# REA, hands-on: install it, connect it to your agent, and crack your first binary (with receipts)

*By Richard (@P7RIM3) · October 13, 2026 (ET)*

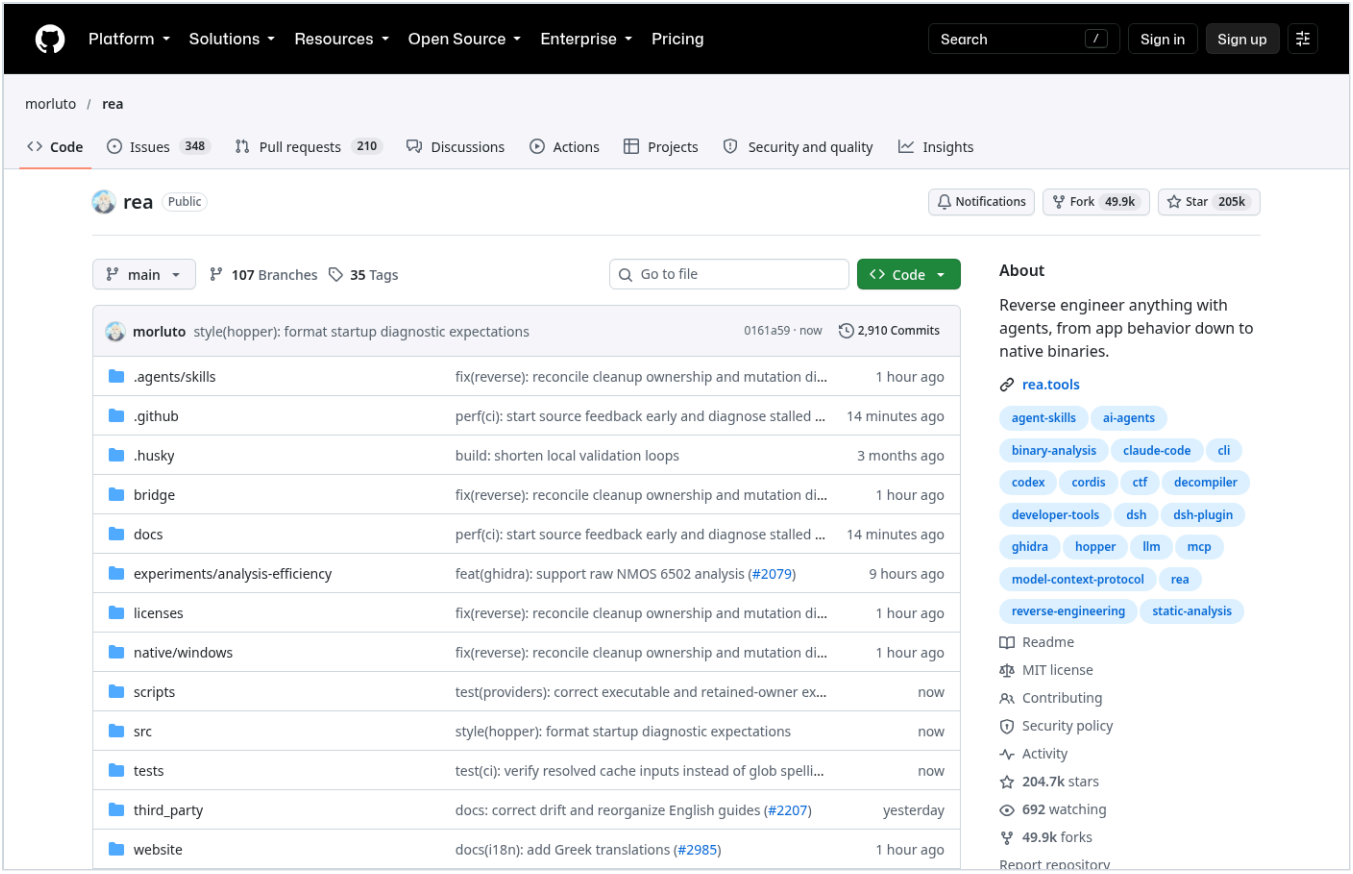



Cover: REA, hands-on. Composite of a real REA terminal capture from this article.

> **About the receipts.** Everything below was checked against the official repo morluto/rea (clone at commit `f104500`, Oct 13, 2026 12:57 PM ET) and the published npm package **rea-agents 6.4.0** on **October 13, 2026 (ET)**. Every terminal image is real output I captured on my Grok Bot box (Debian 13, x64) running REA 6.4.0 with Ghidra 12.1.4 and JDK 21. Each image shows the exact command, and the raw JSON is saved next to it. Every web image is a real screenshot of the live page. The cover is the one composite, built from one of those real captures. I didn't test anything on Windows. The Windows steps are quoted from REA's docs and labeled that way.

---

# 1. The hook




The official morluto/rea repository on GitHub, Oct 13, 2026

You find a feature in an app and want to know how it really works. The usual answer is "read the source." Most of the time there's no source to read. You have a binary, an Electron bundle, an APK, a web page or a firmware image.

**REA ("Reverse Engineer Anything")** is an open-source, MIT-licensed MCP server and CLI that gives a coding agent a real reverse-engineering toolbox. The agent can open software it didn't write, trace a feature through it, and come back with **evidence**: addresses, bytes, pseudocode and call sites, with every claim marked as an observation, an inference or an unknown.

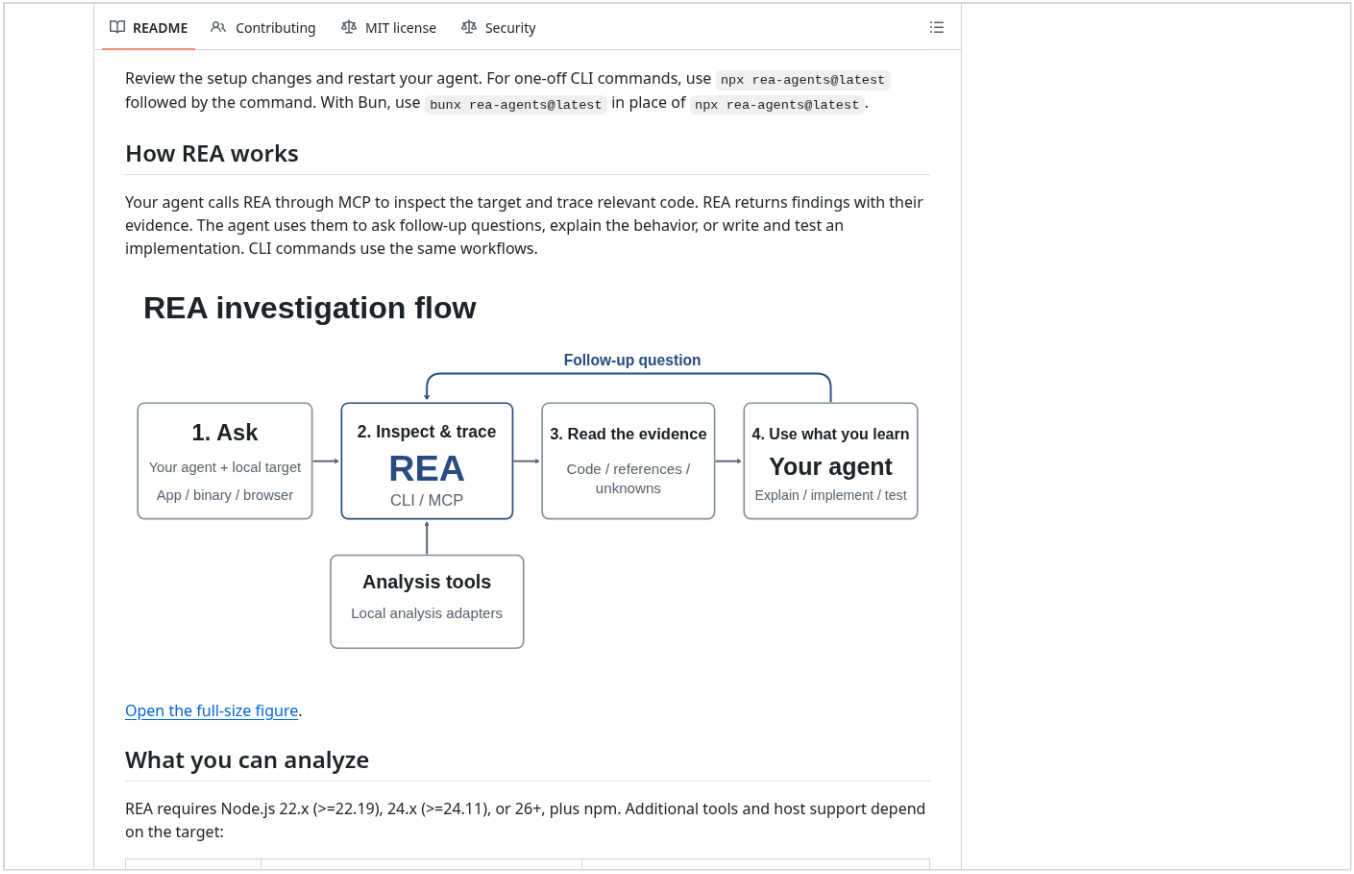
It's also one of the most-starred repos on GitHub right now. The GitHub API showed **204,859 stars** when I checked at 6:28 PM ET today. Stars aren't receipts, though, so I installed REA, pointed it at a practice binary, and wrote down exactly what happened. This is the guide I wish I'd had: every command, every output, and every place where the posts about REA don't match its docs.

Links: repo https://github.com/morluto/rea · website https://rea.tools/ · npm https://www.npmjs.com/package/rea-agents · release notes https://github.com/morluto/rea/releases/tag/rea-agents-6.4.0

---

# 2. What REA is: ask → inspect and trace → read the

## evidence → use it




REA's own "How REA works" diagram in the README

REA's README sums it up in one line: *"See a feature you like. Understand how it works, down to the binary level."* In practice, it's a loop with four steps.

1. **Ask.** You give your agent a specific, testable question about a local target, like "How does this binary decide the password is correct?" A vague "reverse this app" gets vague results.
2. **Inspect and trace.** The agent calls REA tools over MCP: `open_binary`, `search_strings`, `trace_feature`, `xrefs`, `procedure_pseudo_code` and `read_bytes`, out of roughly 170 tools in 6.4.0. REA drives a real analysis engine underneath (Ghidra, Hopper or IDA for native code; its own static analyzers for JavaScript, Electron, .NET, EVM and more). README: *"Analysis runs locally, and results include the evidence and limitations behind each conclusion."*
3. **Read the evidence.** Every result is an **Evidence record** with a content-addressed ID (`ev_…`), the target's SHA-256, the provider and version, a `confidence` (`observed`, `derived` or `inferred`) and a list of `limitations`. Gaps become **residual unknowns** (`unk_…`) that you can record, list and resolve later.
4. **Use what you learn.** Explain the behavior, write your own implementation from a behavioral spec, write tests, or flag a security issue. REA's showcases do exactly this. One is a DX-Ball sound-pan reconstruction that, per the README, *"passes 3,205 original-x86 cases and reproduces all 63 compiled function bytes."*

**A 60-second glossary for absolute beginners:**

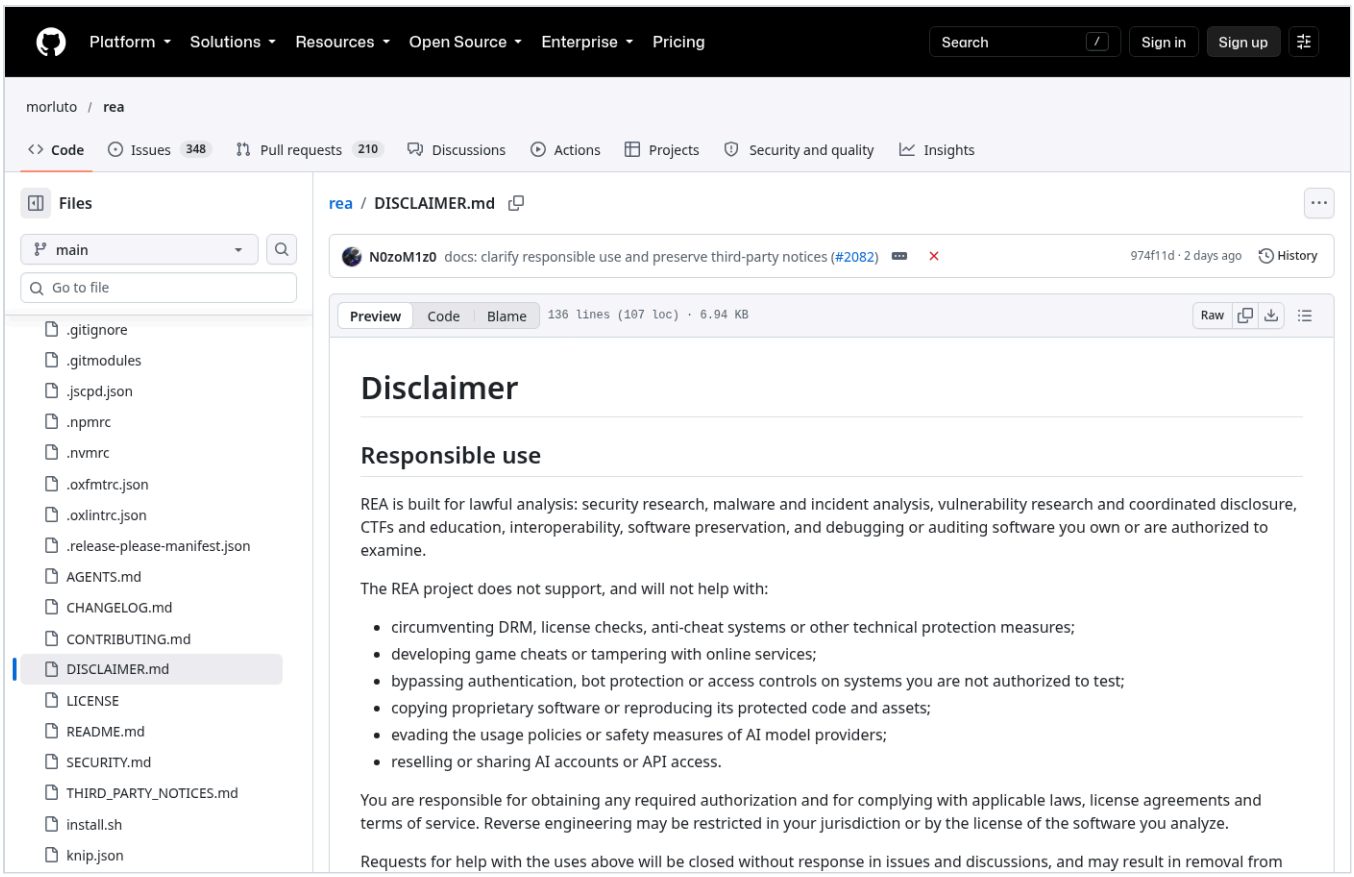
- **Binary / executable:** the compiled program your computer runs (`.exe` on Windows, ELF on Linux, Mach-O on macOS). It's machine code, not readable source.
- **Disassembler / decompiler:** tools that turn machine code back into assembly (disassembler) or C-like **pseudocode** (decompiler). Ghidra, Hopper and IDA do both.
- **String:** readable text stored inside a binary, like error messages and prompts. Strings are usually the fastest clue to where a feature lives.
- **Cross-reference (xref):** a place in the code that uses an address, such as "this instruction loads that string." Following xrefs is how you get from visible text to the code behind it.
- **MCP (Model Context Protocol):** the standard that lets an AI agent call external tools. REA is an MCP server, so your agent can call `trace_feature` the same way it calls a web search.
- **Evidence record (`ev_…`):** REA's receipt for one result. It carries the target hash, tool, parameters, result, confidence and limitations.
- **Residual unknown (`unk_…`):** a question REA (or you) recorded because the evidence doesn't settle it yet.
- **Static vs runtime:** *static* reads files without running them; *runtime* runs or observes the program. REA keeps the two strictly apart.

The same workflows are available from your terminal through the `rea` CLI. I use both below: first the CLI, so you can see every raw output, then the same investigation through my agent over MCP.

A quick note on privacy, from the README FAQ: *"REA analyzes targets locally. Your agent receives the tool results, and its model provider has its own data policy."* So REA doesn't upload your target, but whatever results your agent reads go to your model provider.

---

## 3. Legal and ethical limits, in REA's own words




REA's DISCLAIMER.md on GitHub

Read this section before you touch any tool. REA's DISCLAIMER.md is unusually clear, so I'll quote it directly.

> *"REA is built for lawful analysis: security research, malware and incident analysis, vulnerability research and coordinated disclosure, CTFs and education, interoperability, software preservation, and debugging or auditing software you own or are authorized to examine."*

It also lists what the project **will not help with**:

> *"circumventing DRM, license checks, anti-cheat systems or other technical protection measures; developing game cheats or tampering with online services; bypassing authentication, bot protection or access controls on systems you are not authorized to test; copying proprietary software or reproducing its protected code and assets; evading the usage policies or safety measures of AI model providers; reselling or sharing AI accounts or API access."*

Three more lines are easy to miss and matter most in practice:

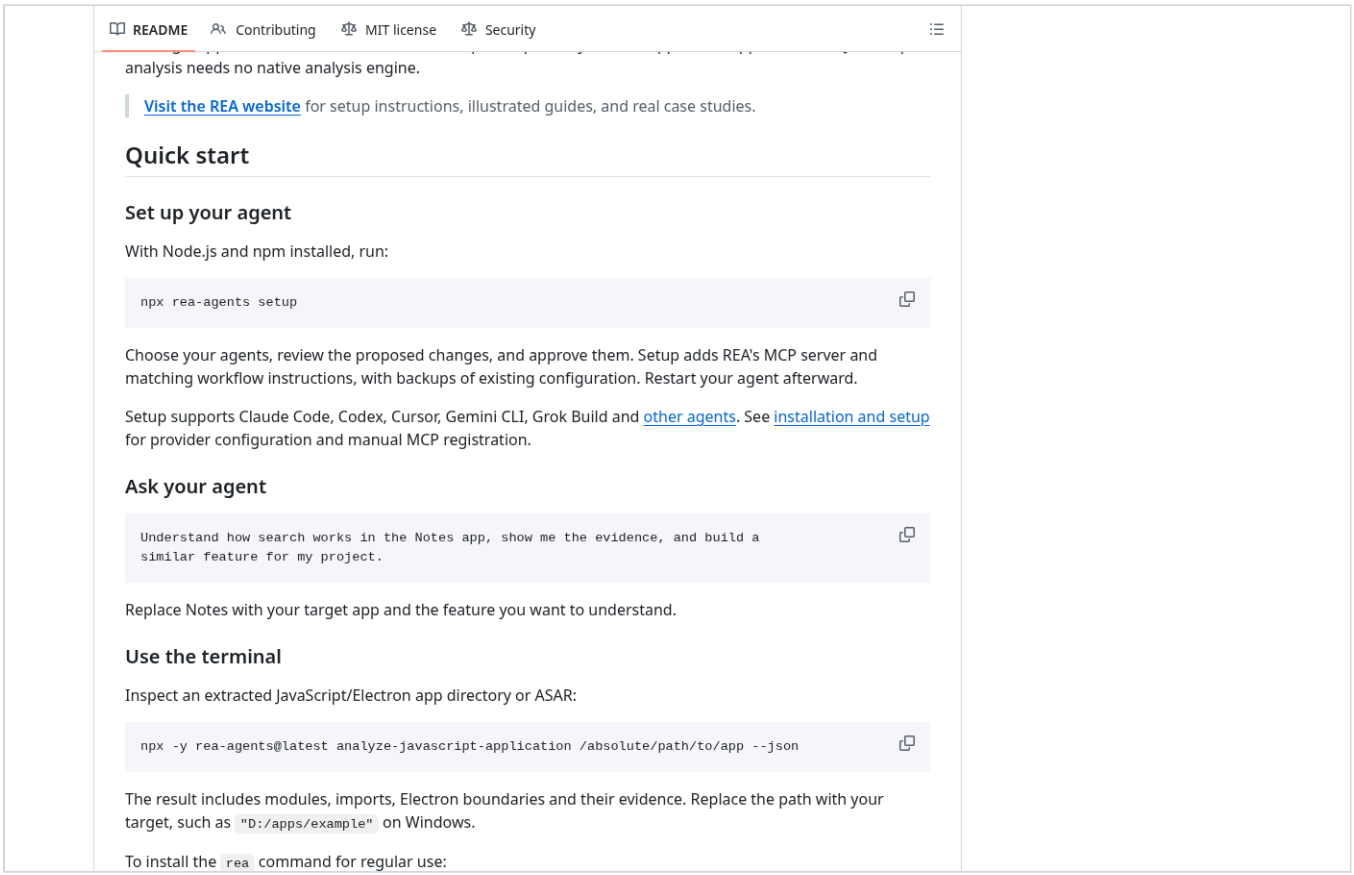
> *"Public availability of software, possession of a file, or access to a debugging endpoint does not by itself establish permission for every operation."*
>
> *"Authorization to inspect an artifact does not necessarily authorize execution, interception, device changes, or interaction with another party's system."*
>
> *"Obtaining source-like output or an analysis report does not grant rights to the original program, its data, or the generated output."*

On tools: you must legally own any commercial engine you connect (IDA Pro and Hex-Rays, Hopper, JEB). REA *"provides no commercial-tool license, key, or activation entitlement."* Ghidra is free and open source, which is why I use it here.

My rule for this guide: **every target is something I built myself.** That's the practice crackme, a tiny Electron-style app, a local web page and a hand-assembled EVM contract. That's the safest way to learn.

---

## 4. Step-by-step install (Linux and Windows)




README "Quick start": one command, npx rea-agents setup

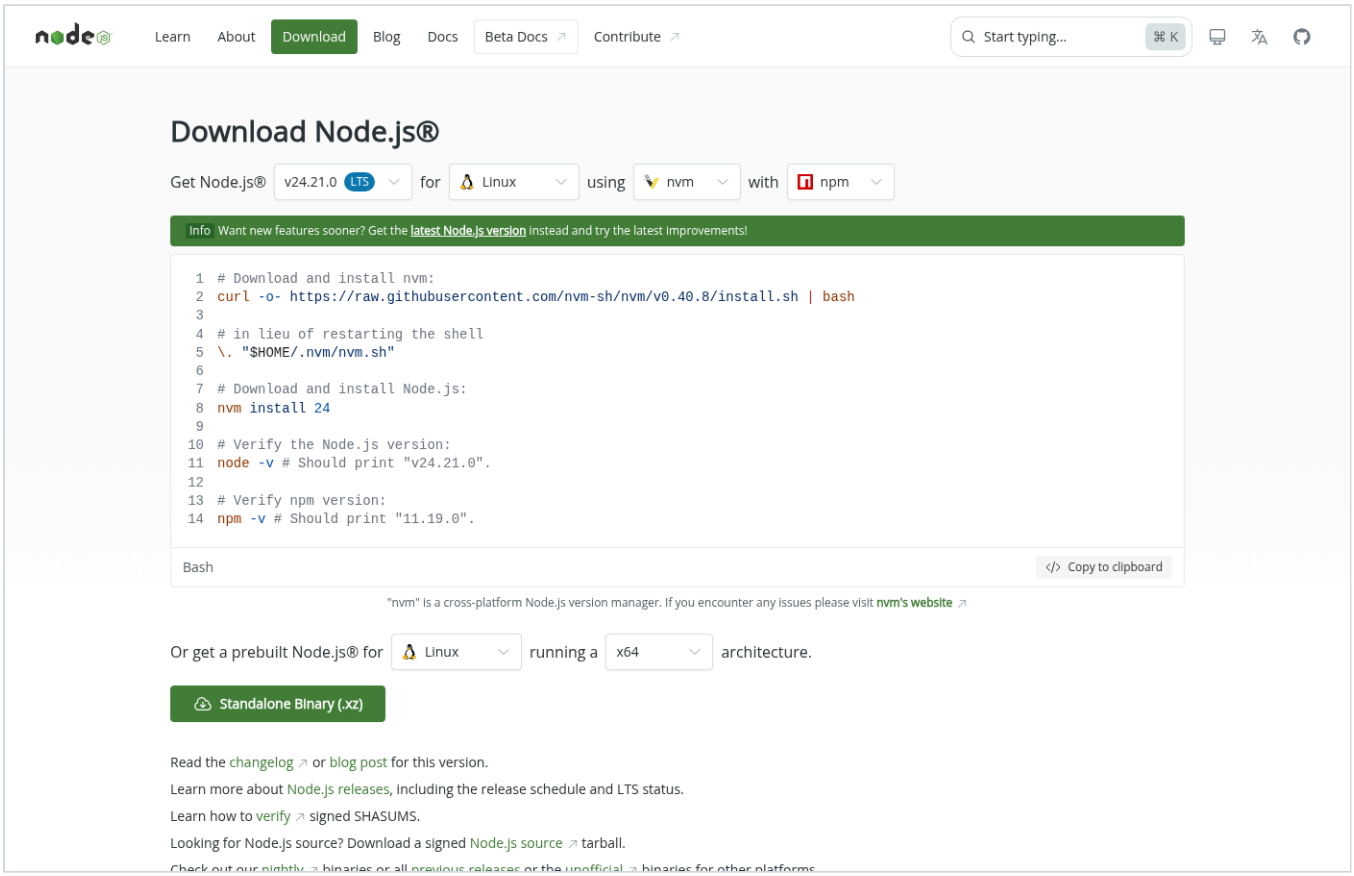
**What you need before you start (checklist):**

- A computer running Linux x64/arm64, macOS, or Windows 10+ x64 (Windows native analysis is experimental, see §4.4).
- **Node.js** 22.19+, 24.11+ or 26+ with npm (§4.1).

- An **agent** that supports local MCP servers: Claude Code, Cursor, Codex, Gemini CLI and so on, or Grok Bot through its hosted connector (§5).
- *For native binaries only:* **Ghidra 12.1.x + a 64-bit JDK 21+** (free), or Hopper/IDA if you already own a license.
- *For Android only:* headless JADX + JDK. *For firmware only:* Binwalk/Unblob on Linux.
- A **target you're allowed to analyze.** Build your own, use a CTF binary, or use software you own and are authorized to examine (§3).

REA has two layers: the **REA package** (MCP server, CLI and agent skill) and an optional **native analysis engine** for binaries. Static JavaScript/Electron, .NET and EVM analysis need no engine at all. Native binaries need Ghidra, Hopper or IDA.

## 4.1 Prerequisite: Node.js in a supported range




nodejs.org download page

The README says: *"REA requires Node.js 22.x (>=22.19), 24.x (>=24.11), or 26+, plus npm."* The installation guide adds that *"Node.js 23, 25, and prereleases are unsupported."* So "Node 22 or newer," as some posts put it, isn't quite right. Node 23 or 25 won't work. Get an LTS build from https://nodejs.org/en/download.

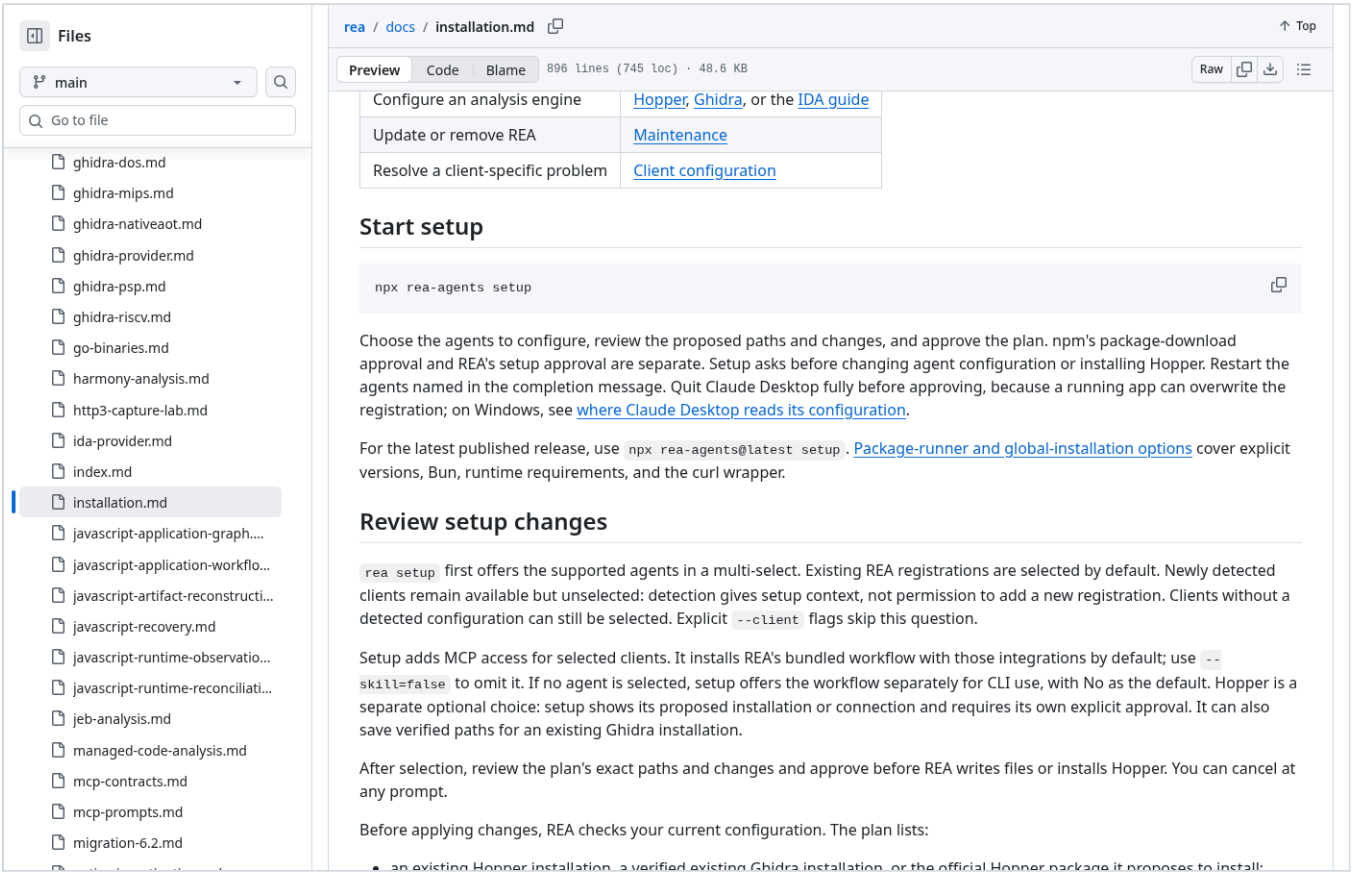
My box had Node 20 on the default PATH, which isn't supported, so I put a Node 24 build first on PATH. Here's what I verified before going further:




Real capture: node, npm, java, Ghidra path, rea and npm versions

`npm view rea-agents dist-tags.latest` printing `6.4.0` is the quickest way to confirm which release you're getting.

## 4.2 Linux, step by step

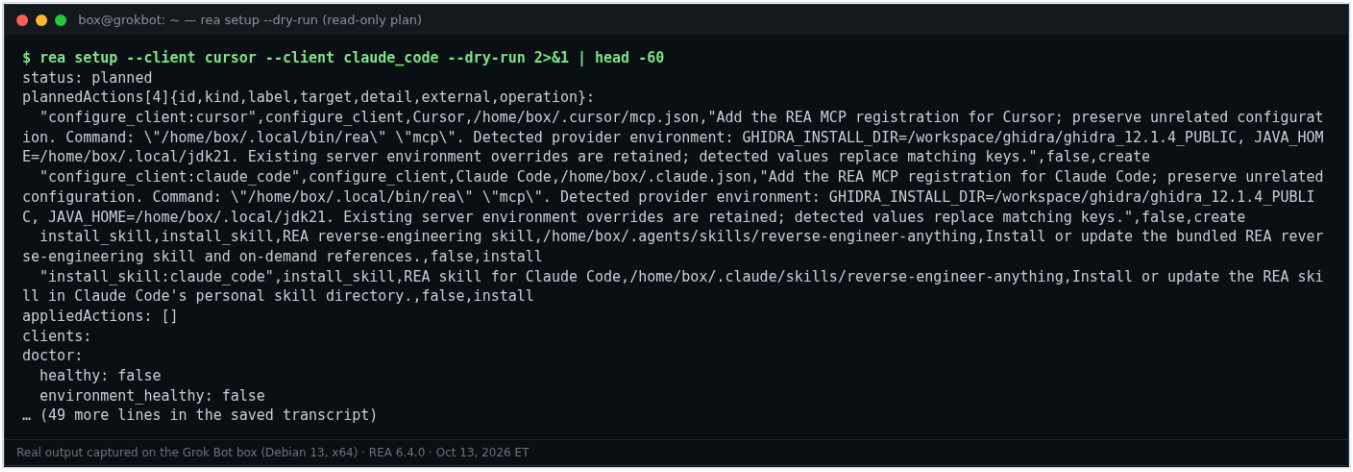



REA docs: "Start setup", the reviewed setup transaction

1. **Check Node:** `node --version` must print v22.19+, v24.11+ or v26+.
2. **Run guided setup:** `npx rea-agents setup`. The docs say: *"Choose the agents to configure, review the proposed paths and changes, and approve the plan."* npm's download prompt and REA's own approval are separate. Setup *"asks before changing agent configuration or installing Hopper"* and makes backups of existing config. To pin the newest release, use `npx rea-agents@latest setup`.
3. **Restart the agents** named in the completion message.

4. **(Optional) Install the CLI for regular use:** `npm install --global rea-agents`, then `rea --help`. The docs also describe an optional curl wrapper (`curl -fsSL https://raw.githubusercontent.com/morluto/rea/main/install.sh | bash`). It only installs the global npm package, and `bash -s -- --dry-run` previews what it would do.
5. **(Optional, for native binaries) Add Ghidra.** REA *"connects to an existing Ghidra installation"*. It never downloads Ghidra or Java for you. See §4.3.
6. **Check readiness for your actual task, not everything:** `rea doctor --provider ghidra --json`. Without a scope, `rea doctor` audits every integration, and *"its overall `healthy` value can be false while your chosen workflow works."*

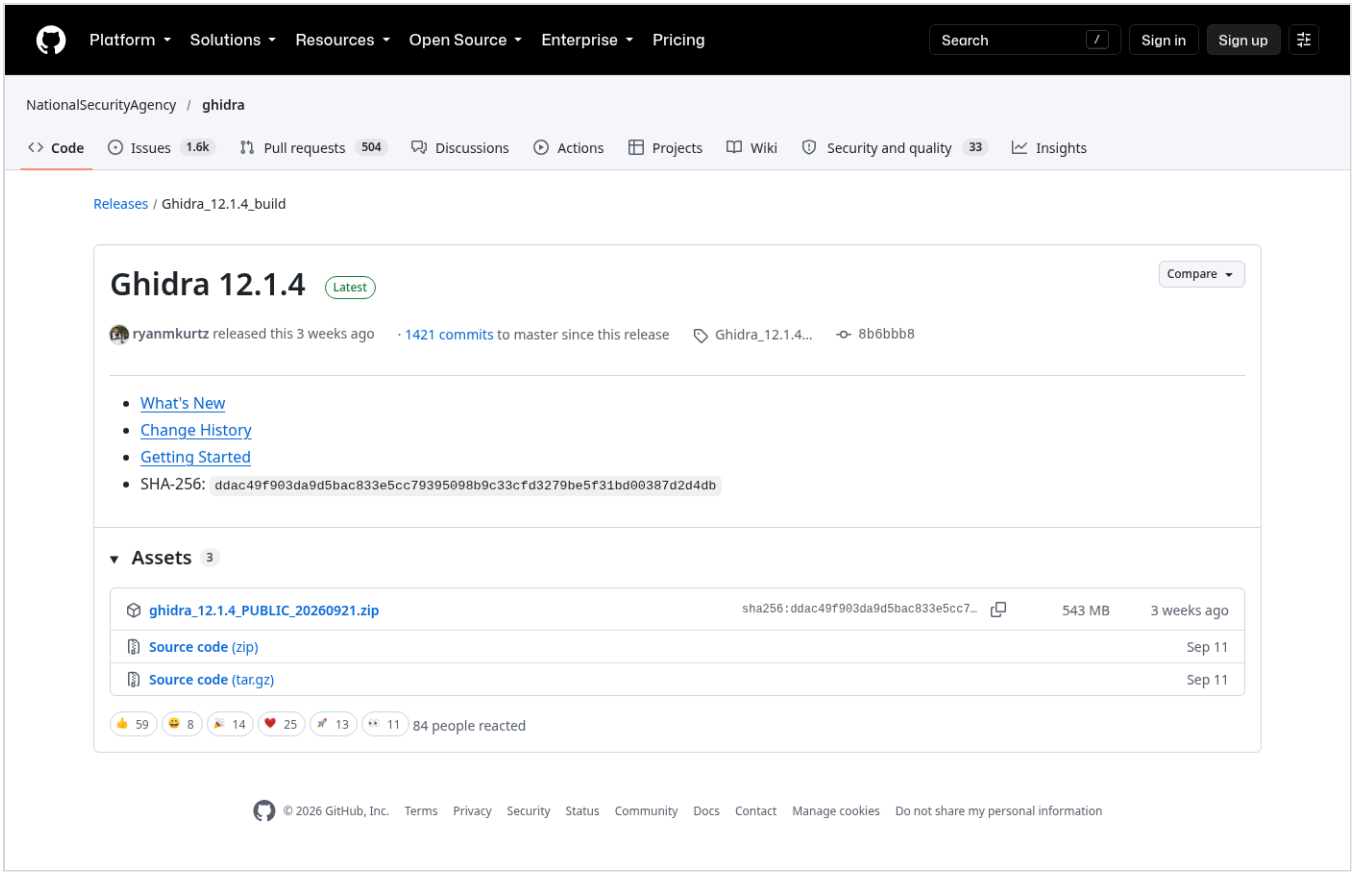
If you want to see what setup would change before it touches anything, there's a read-only plan mode. Here's mine for Cursor plus Claude Code: `status: planned`, `appliedActions: []`, nothing written.




Real capture: rea setup --client cursor --client claude_code --dry-run

It shows the exact files it would write: `~/.cursor/mcp.json` and `~/.claude.json` for the MCP registrations, plus the skill under `~/.agents/skills/` and `~/.claude/skills/`. It also picked up my Ghidra and JDK paths to add to the registration environment.

## 4.3 Ghidra on Linux (what I ran)




Ghidra 12.1.4 official release page (NationalSecurityAgency/ghidra)

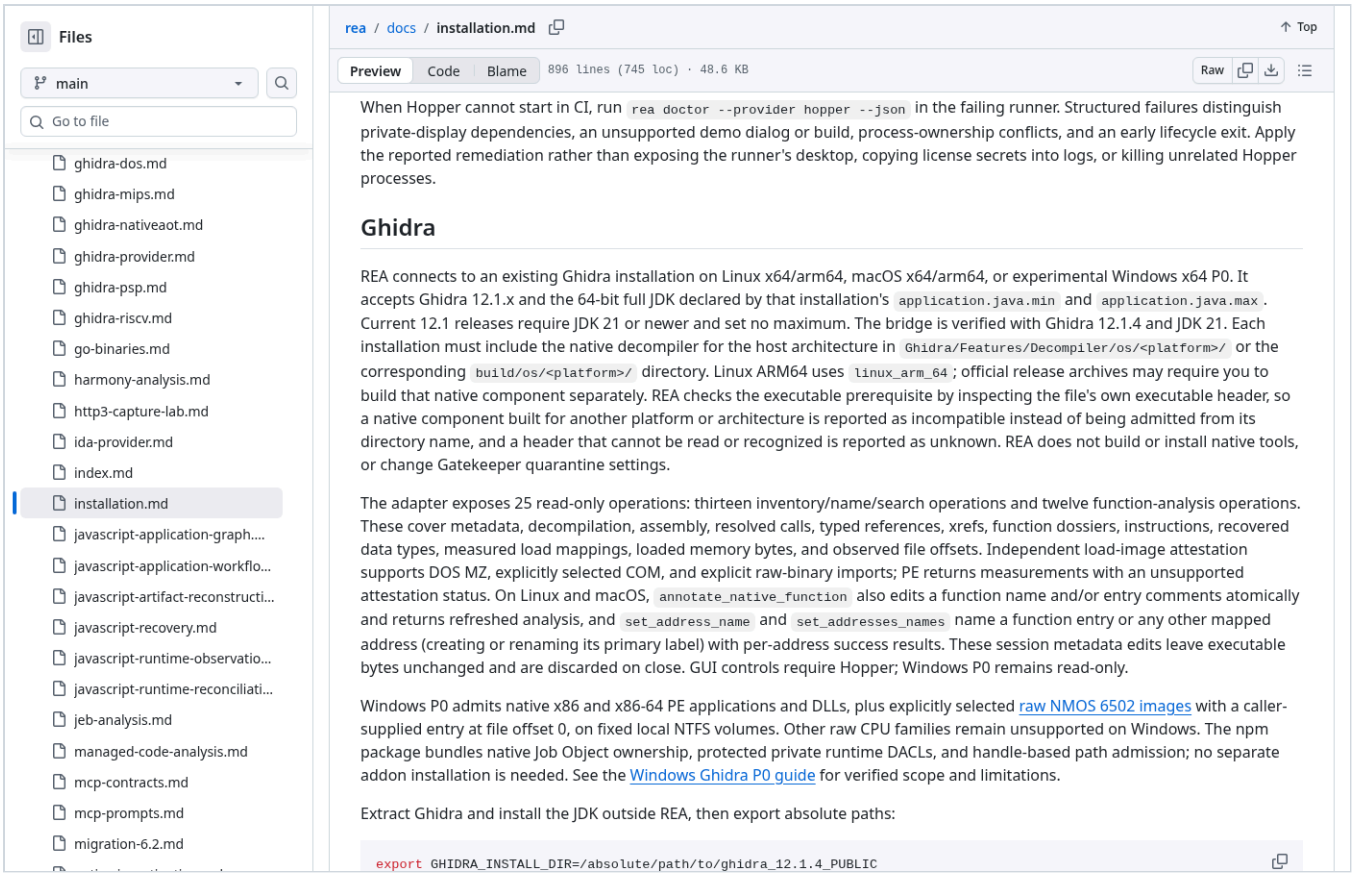
REA's docs: *"It accepts Ghidra 12.1.x and the 64-bit full JDK declared by that installation... Current 12.1 releases require JDK 21 or newer... The bridge is verified with Ghidra 12.1.4 and JDK 21."*




Adoptium Temurin JDK downloads

1. Install a **64-bit full JDK 21+** (for example Temurin from https://adoptium.net/temurin/releases/?version=21).
2. Download **Ghidra 12.1.4** only from the official release page, https://github.com/NationalSecurityAgency/ghidra/releases/tag/Ghidra_12.1.4_build, and extract it.
3. Export the paths exactly as the docs show:

```
export GHIDRA_INSTALL_DIR=/absolute/path/to/ghidra_12.1.4_PUBLIC
export JAVA_HOME=/absolute/path/to/jdk-21 # optional if java/javac are on PATH
rea doctor --json
rea setup
```




REA docs: the Ghidra provider section

On my box, Ghidra lives at `/workspace/ghidra/ghidra_12.1.4_PUBLIC`. The scoped check came back fully healthy: version, headless launcher, native decompiler (`reports elf x86_64, which matches linux/x64`) and Java.

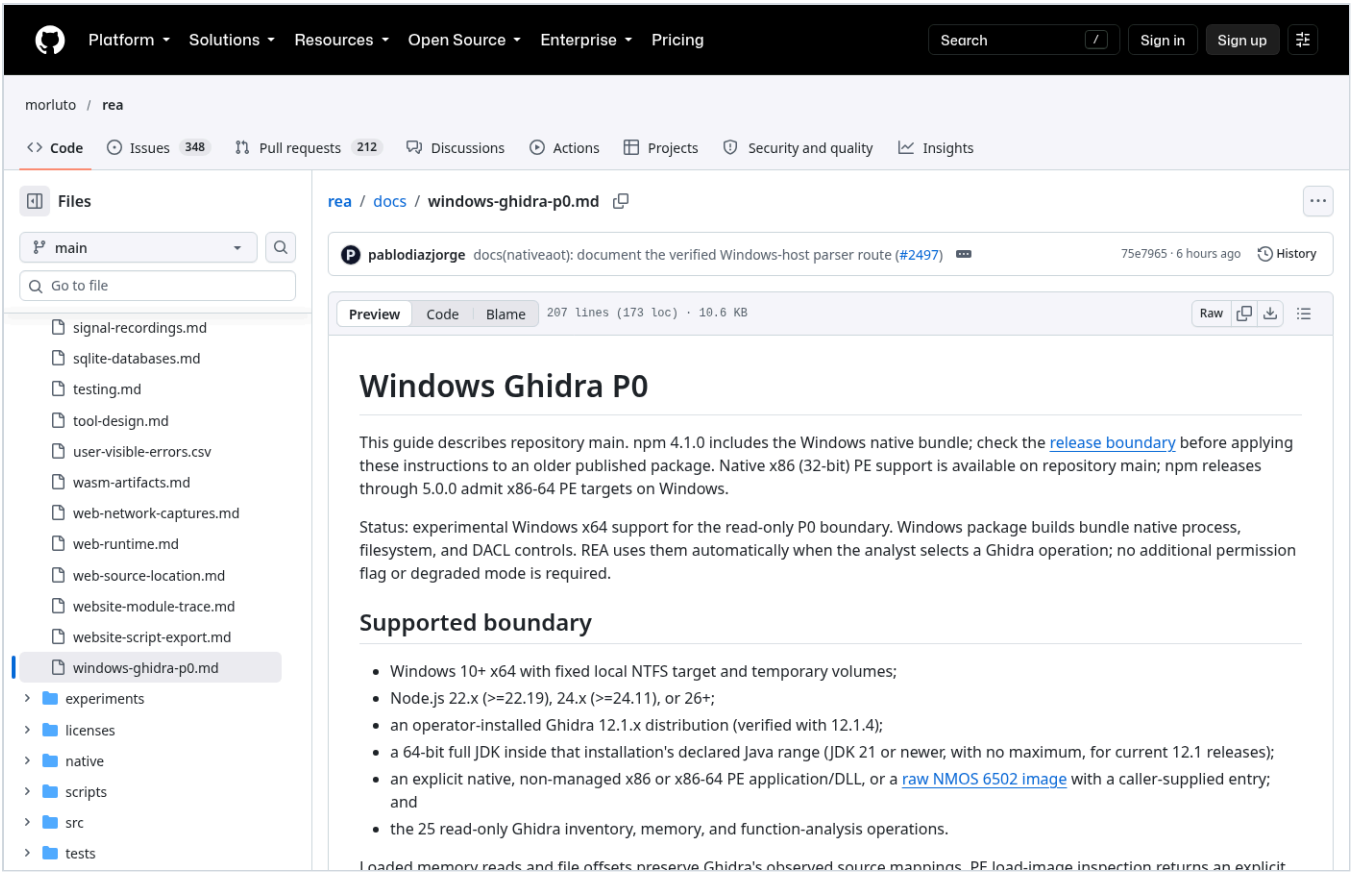



Real capture: rea doctor --provider ghidra, all nine checks healthy

One honest note: an unscoped `rea doctor` on this box flags `host: debian 13` as `unsupported_host`. That list (*"macOS 12+, Ubuntu 24.04+, Fedora 41+..."*) is about REA's automated Hopper installer and supported-host matrix. The Ghidra path still worked end to end on Debian 13, as you'll see. I'm reporting what I observed, not claiming Debian is officially supported.

How REA treats your files with Ghidra (docs): each session uses *"an ephemeral temporary project,"* passes `-readOnly` and `-deleteProject`, and *"never opens an existing user project."* Annotations only change the session database; *"original executable bytes are never written"* (that's from the tool's own `limitations` field).

## 4.4 Windows, step by step (quoted from the docs; I didn't test it)




REA docs: experimental Windows Ghidra P0

1. Install Node.js **22.19+, 24.11+ or 26+** with npm.
2. In a normal terminal (*not* one started by Claude Desktop, see below), run `npx rea-agents setup`, pick your agents, review the plan, approve, then restart them. Per the docs, *"On Windows, setup wraps either command in `cmd.exe /d /c` so clients can start the runner's command shim."*
3. Optionally install the CLI: `npm install --global rea-agents`, then `rea --help`. Windows paths work in commands, for example `"D:/apps/example"` (README).
4. For native binaries, install a 64-bit JDK 21+, extract Ghidra 12.1.4, and set the paths in **PowerShell** exactly as documented:

```
$env:GHIDRA_INSTALL_DIR = "C:\tools\ghidra_12.1.4_PUBLIC"
$env:JAVA_HOME = "C:\Program Files\Java\jdk-21"
rea doctor --json
rea providers --json
```

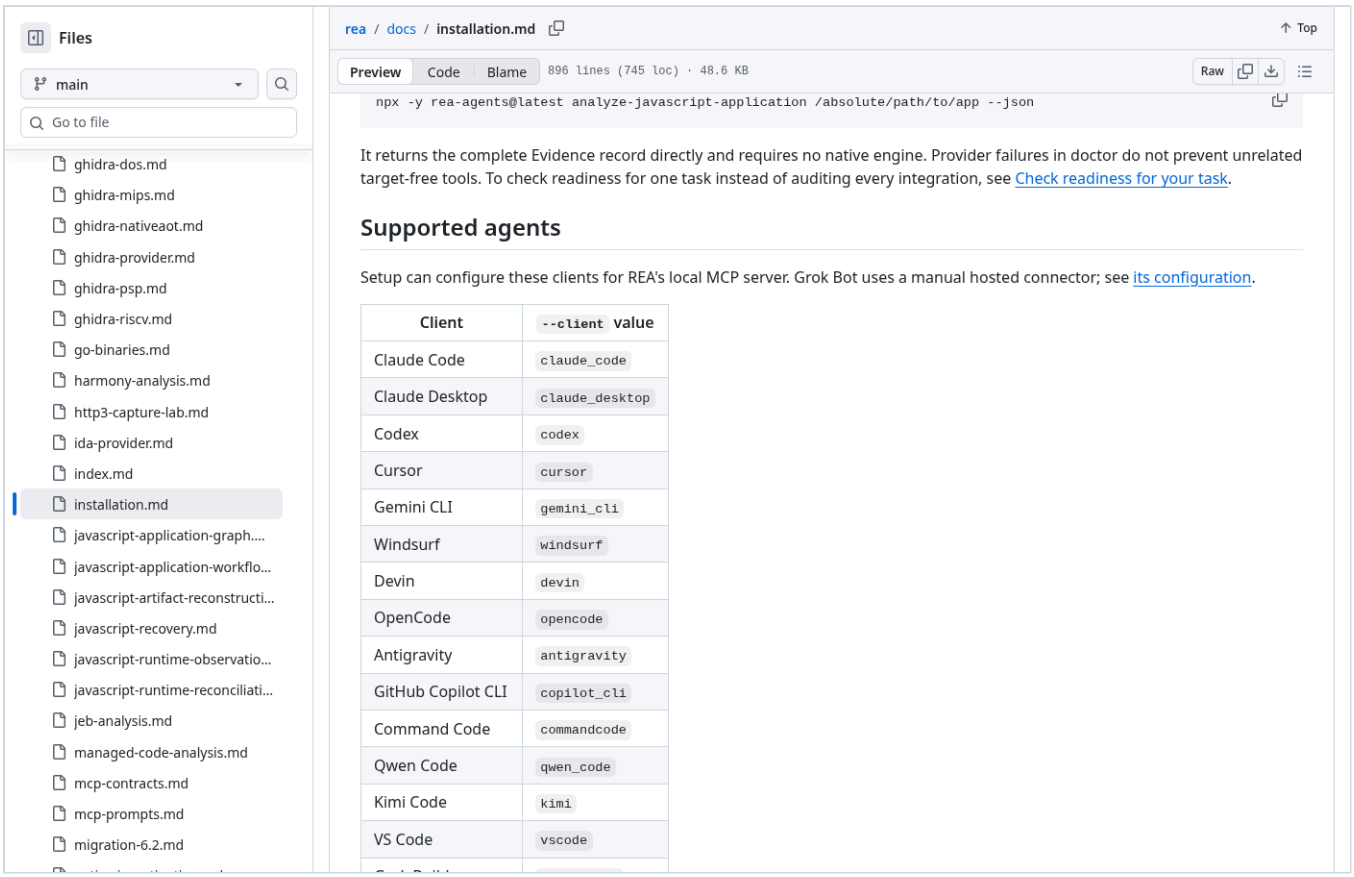
1. Run `rea setup` again so the verified Ghidra/JDK paths are saved into your agent registrations (*"It preserves valid detected Ghidra/JDK settings in agent registrations."*)

Know the Windows limits before you start. Windows Ghidra support is **experimental "P0"**: Windows 10+ x64, *"fixed local NTFS target and temporary volumes,"* native **x86/x86-64 PE** EXEs and DLLs (plus raw 6502 images), and *"the 25 read-only Ghidra inventory, memory, and function-analysis operations."* Renaming and annotating functions (`annotate_native_function`) is Linux/macOS only, and *"Hopper installation remains unavailable on Windows."* If you use **Claude Desktop** on Windows, *"Quit Claude Desktop fully before approving,"* because the MSIX

build can overwrite the registration. Full guide: [https://github.com/morluto/rea/blob/main/docs/windows-ghidra-p0.md](https://github.com/morluto/rea/blob/main/docs/windows-ghidra-p0.md).

---

## 5. Connect REA to your agent: Grok Bot, Claude Code or Cursor



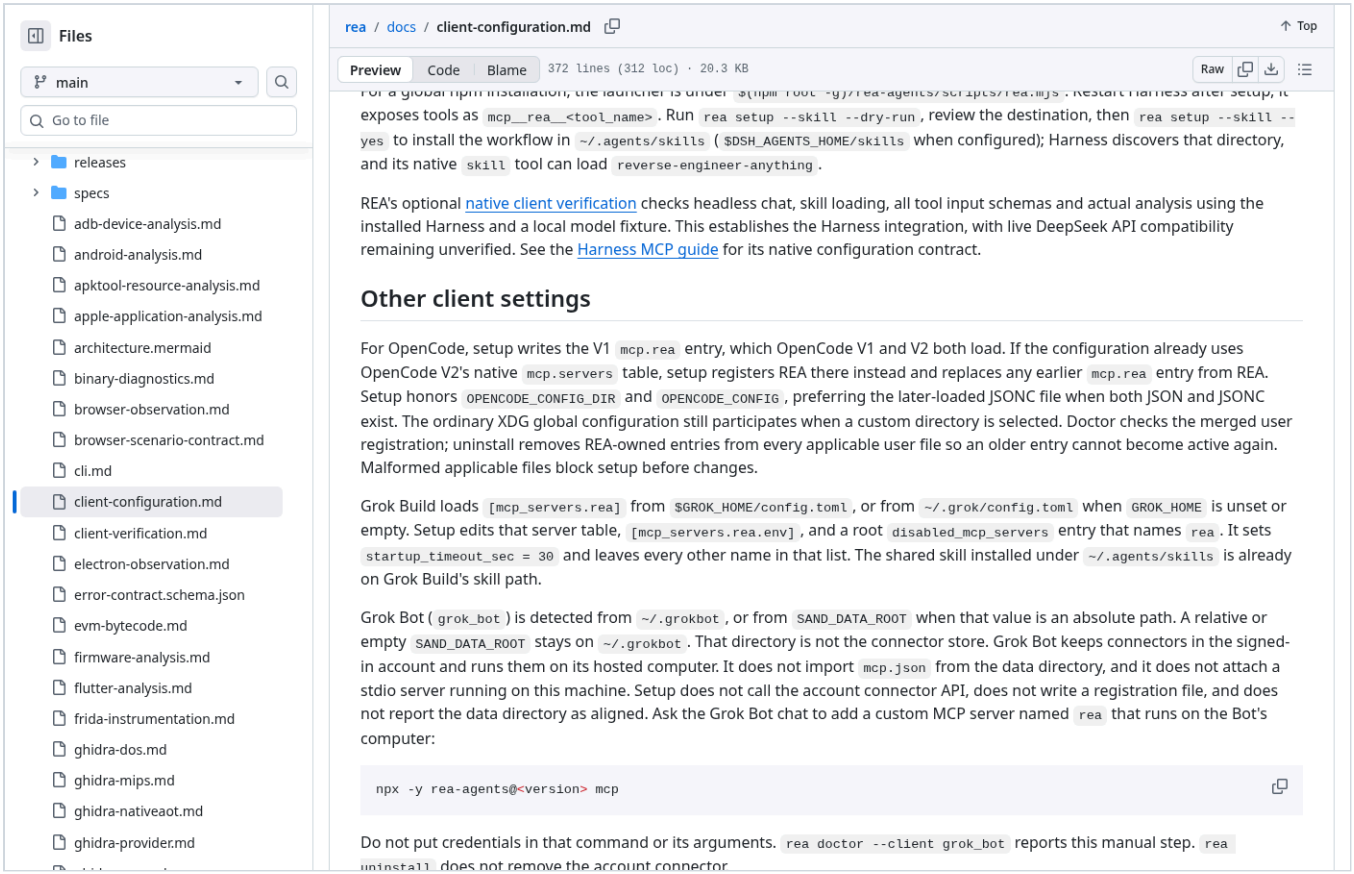

REA docs: the supported-agents table with --client IDs

The docs say the npm 6.4.0 package accepted setup plans for 17 client IDs (`main` lists 23). The three I care about are documented like this.

| Agent | How REA documents it | What it writes |
|---|---|---|
| **Claude Code** | `npx rea-agents setup` and select Claude Code, or `rea setup --client claude_code` | `mcpServers.rea` in `~/.claude.json`, plus the skill in Claude Code's personal skill dir (my dry-run: `~/.claude/skills/reverse-engineer-anything`) |
| **Cursor** | `npx rea-agents setup` and select Cursor, or `rea setup --client cursor` | `~/.cursor/mcp.json` (my dry-run), plus the shared skill |
| **Grok Bot** | **Manual hosted connector.** Setup doesn't write anything for it | You ask Grok Bot to add a custom MCP server named `rea` |

Restart the agent after setup. To check a single registration, run `rea doctor --client claude_code --json` (or `cursor`).

## Grok Bot: the one that works differently




REA docs: Grok Bot connector instructions in client-configuration.md

REA's client docs say: *"Grok Bot keeps connectors in the signed-in account and runs them on its hosted computer. It does not import `mcp.json` from the data directory, and it does not attach a stdio server running on this machine."* The documented step is: *"Ask the Grok Bot chat to add a custom MCP server named `rea` that runs on the Bot's computer"* with:

```
npx -y rea-agents@<version> mcp
```

Use an exact version, for example `rea-agents@6.4.0`, and *"Do not put credentials in that command or its arguments."* `rea doctor --client grok_bot` reports this as a manual step, so a "not registered" result there is expected:




Real capture: rea doctor --client grok_bot explains the manual connector step

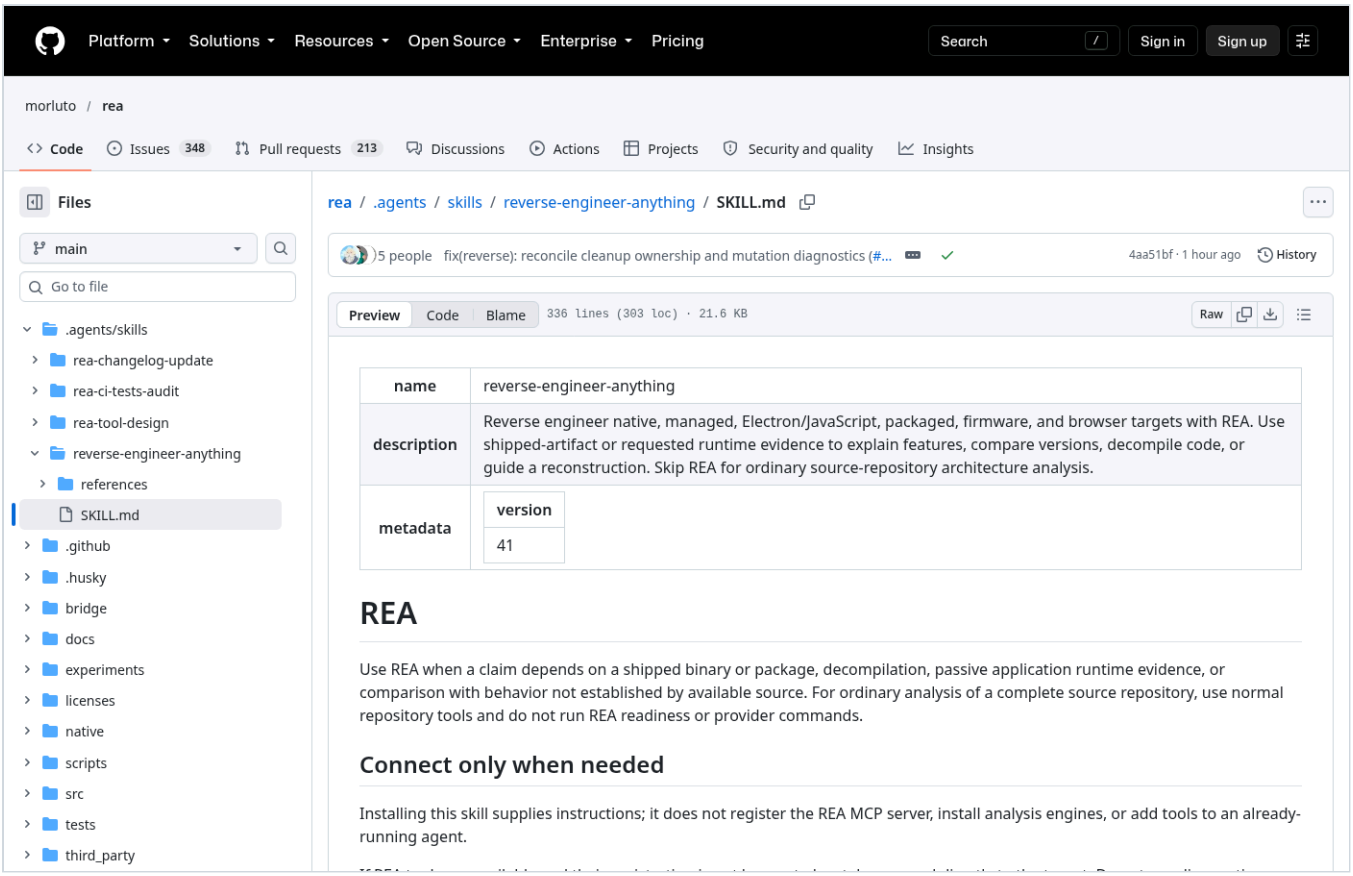
On my account, the `rea` connector runs on the Bot's computer (the same box as everything above), and section 8 shows real tool calls through it.

For any other MCP client, the documented manual registration is:

```
{ "mcpServers": { "rea": { "command": "npx", "args": ["-y", "rea-agents@6.4.0", "mcp"] } } }
```

---

# 6. Route the target first: REA's official agent skill, as a step-by-step



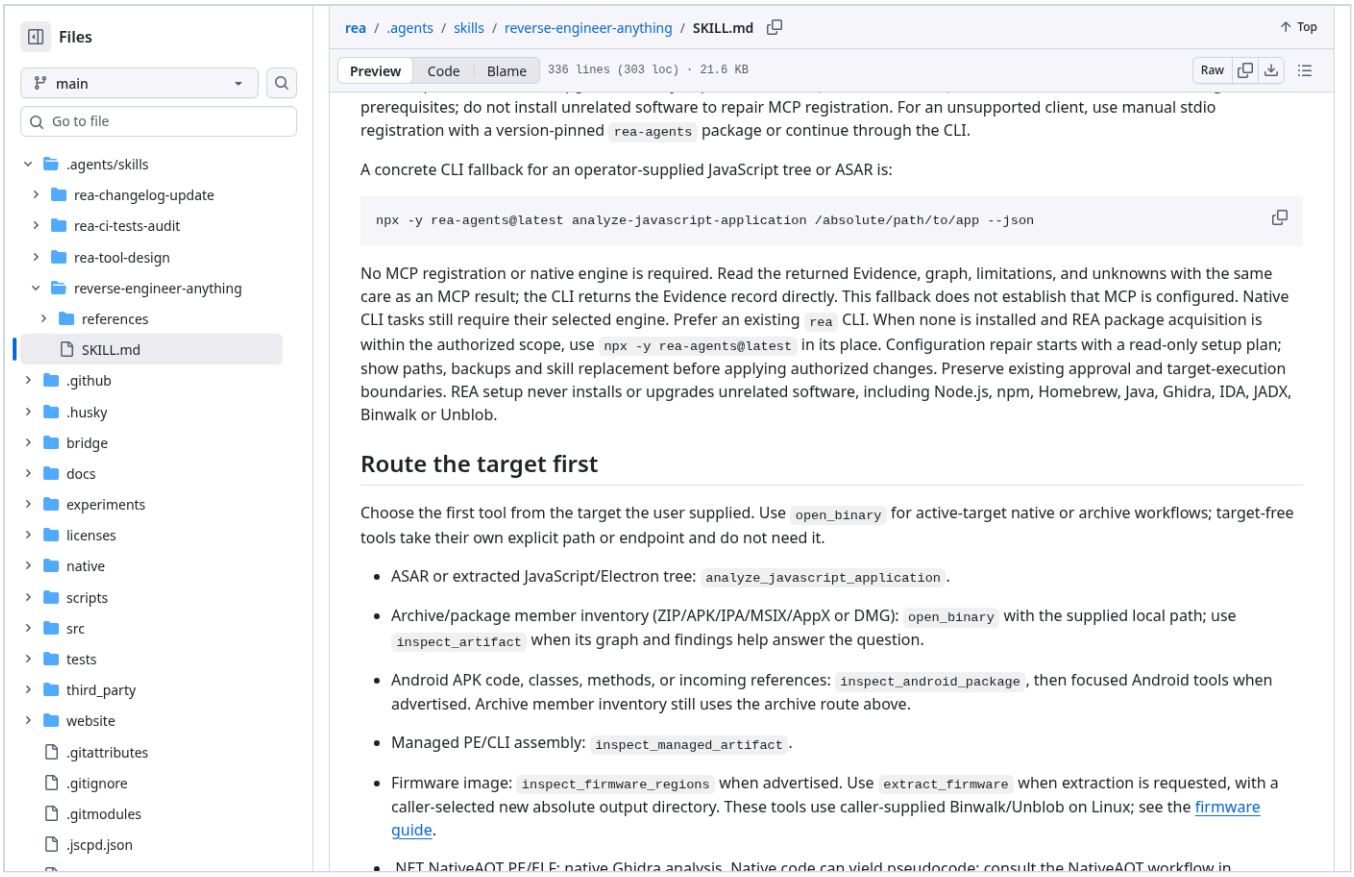

The official REA skill, .agents/skills/reverse-engineer-anything/SKILL.md, on GitHub

Setup also installs REA's **agent skill**, the instructions your agent follows: `.agents/skills/reverse-engineer-anything/SKILL.md`. It's the best guide to *how to use* REA, and it's written for the agent, not for marketing. Some viral posts present these as secret prompts "in the repo the whole time." They're documented, and you should read them.

**Step 1: decide whether you need REA at all.** The skill: *"Use REA when a claim depends on a shipped binary or package, decompilation, passive application runtime evidence, or comparison with behavior not established by available source. For ordinary analysis of a complete source repository, use normal repository tools."*

**Step 2: connect only when needed.** *"If REA tools are available and their registration is not known to be stale, proceed directly to the target. Do not run diagnostics or setup before every investigation."* When tools are missing: *"Prefer an existing `rea` CLI. When none is installed and REA package acquisition is within the authorized scope, use `npx -y rea-agents@latest` in its place."* Repairs start *"with a read-only setup plan"* (the `--dry-run` from §4.2).

**Step 3: route the target to its first tool.** *"Choose the first tool from the target the user supplied."* The routing map:




SKILL.md: "Route the target first"

| If your target is… | First tool | Notes from the skill and docs |
|---|---|---|
| Native executable, library or analysis DB (ELF, PE, Mach-O) | `open_binary` | Then focused tools (`search_strings`, `trace_feature`, `xrefs`, `procedure_pseudo_code`). `binary_overview` when useful |
| ASAR or extracted JavaScript/Electron tree | `analyze_javascript_application` | No disassembler. Use `"detail": "summary"` for big apps |
| Archive or package inventory (ZIP/APK/IPA/MSIX/AppX/DMG) | `open_binary`, then `inspect_artifact` | `inspect_artifact` returns the artifact graph and findings |
| Android APK code, classes, methods | `inspect_android_package` | Needs a caller-supplied headless JADX + JDK |
| Managed .NET PE/CLI assembly | `inspect_managed_artifact` | Static only, no engine |
| Firmware image | `inspect_firmware_regions` | Caller-supplied Binwalk (Linux). `extract_firmware` needs a new output dir |
| A browser page you own, already open | `list_browser_targets` | Loopback Chrome CDP |
| Saved HAR / mitmproxy capture | `inspect_web_network_capture` | Never fetches recorded URLs |
| An Electron app you own, already running | `list_electron_targets` | Loopback CDP |
| Recorded packet capture (fields) | `inspect_packet_capture` | **Main branch only.** Not in npm 6.4.0 (see below) |
| Packet summaries (DNS/SNI/endpoints) | `inspect_pcap_capture` | **Main branch only.** Not in npm 6.4.0 |

| If your target is… | First tool | Notes from the skill and docs |
|---|---|---|
| Local EVM bytecode | `inspect_evm_interface` | Offline. Selectors and arguments are *inferred* |

**A receipt the posts miss:** the skill on `main` is **version 41**, but npm 6.4.0 bundles **version 37**. `inspect_packet_capture`, `inspect_pcap_capture`, `inspect_go_binary`, `inspect_wasm_artifact` and `inspect_sqlite_database` appear in the main-branch skill and docs, but the 6.4.0 MCP server connected to my agent **doesn't advertise them**, and the 6.4.0 CLI has no `inspect-go-binary`:




Real capture: inspect-go-binary is not a command in npm 6.4.0

The skill itself warns you: *"Use the connected server's actual tool list and input schemas; a skill installed from repository main may describe capabilities absent from an older npm release."* Trust your server's tool list over any blog post, this one included.

**Step 4: work summary-first.** *"Start with the default result… Make a focused follow-up only when the returned result leaves a specific question unanswered."* And the rule that matters most: *"Every conclusion must distinguish observations, inferences, and unknowns. Cite Evidence IDs… never imply that static analysis observed execution."*

**Step 5: finish cleanly.** *"Close an opened native session with `close_binary` when the investigation is complete."*

---

## 7. Your first investigation: a practice crackme, start to finish




Real capture: the read-only workspace with the target, its SHA-256, and a refused write

**The target.** A tiny, harmless practice program built for this guide: `/workspace/crackme/crackme` (Linux ELF x86-64) and `crackme.exe` (Windows PE x86-64). It asks for a password and prints *"Access granted"* or *"Wrong password."* The goal is to find the password **without reading the source**. It's my own binary, so analyzing and running it is fully authorized.

**Step 0: make a read-only workspace.** I copied the binaries into `/workspace/re/crackme/target/`, ran `chmod -R a-w` on it, and keep all notes in `notes/`. The capture above shows the file hash and that writing to `target/` fails. More on why in §9.

From here on, `T=/workspace/re/crackme/target/crackme`. Each CLI call takes about 10 seconds on this box, because every CLI invocation is a separate process that re-imports the binary into a fresh, ephemeral Ghidra project. The MCP route in §8 keeps one session open instead.

## Step 1: search for the visible text




Real capture: rea search for "password"

```
rea search $T password --provider ghidra --json
```

You get two strings, at `0x102011` and `0x102046`, with `confidence: observed`. Look closely at the first one: `"IAYEnter the password: "`. Those three stray bytes, `IAY`, aren't part of the prompt. Something is stored right before it. Remember that.

## Step 2: trace the failure message to the code that uses it

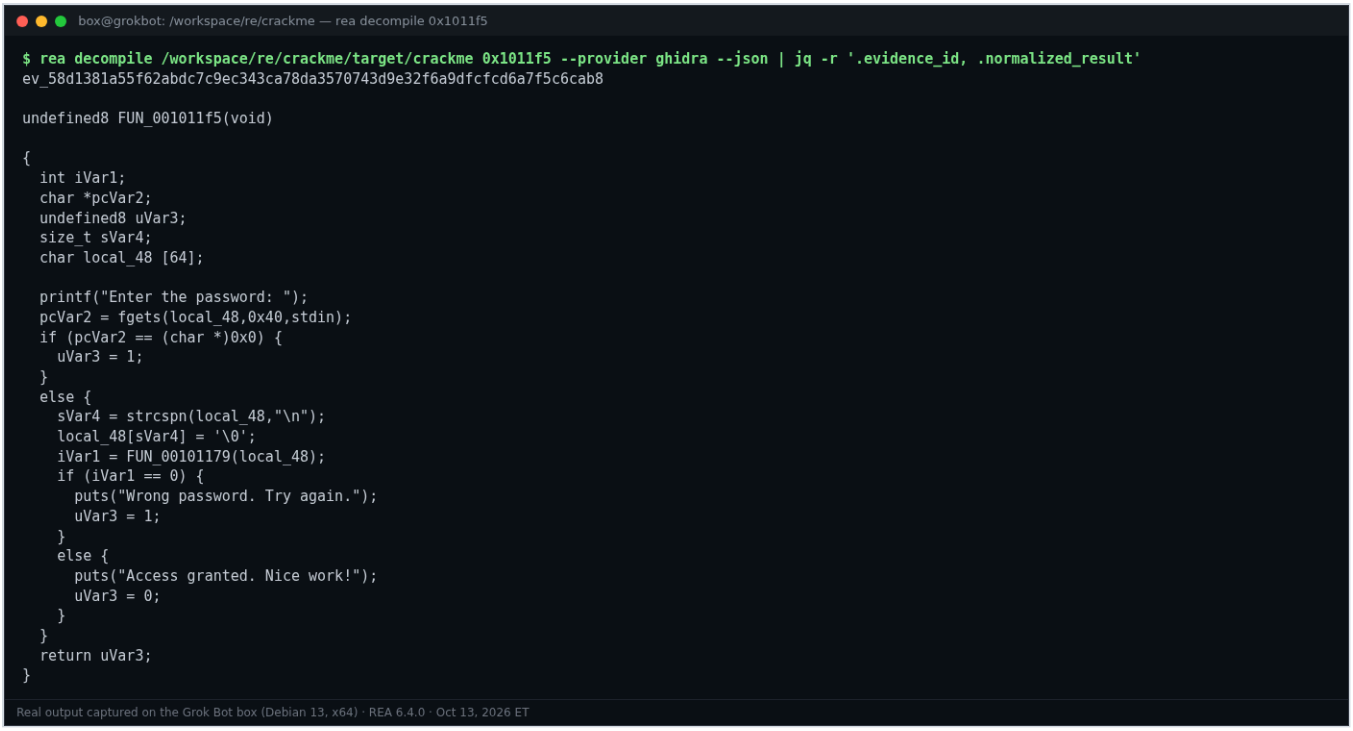



Real capture: rea trace "Wrong password"

```
rea trace $T 'Wrong password' --provider ghidra --json
```

`trace_feature` follows the string through its cross-references. It's used at `0x101276` and `0x10127d`, both inside the function at **`0x1011f5`** (Ghidra calls it `FUN_001011f5`, because the binary has no symbol names). `confidence: derived`: REA composed this from observed facts.

## Step 3: decompile that function




Real capture: rea decompile 0x1011f5, the program's main logic

```
rea decompile $T 0x1011f5 --provider ghidra --json
```

This is `main`. It reads up to 64 bytes with `fgets`, strips the newline, calls **`FUN_00101179(local_48)`** and prints "Access granted" if the result is non-zero. So the real check lives at `0x101179`.

## Step 4: decompile the check



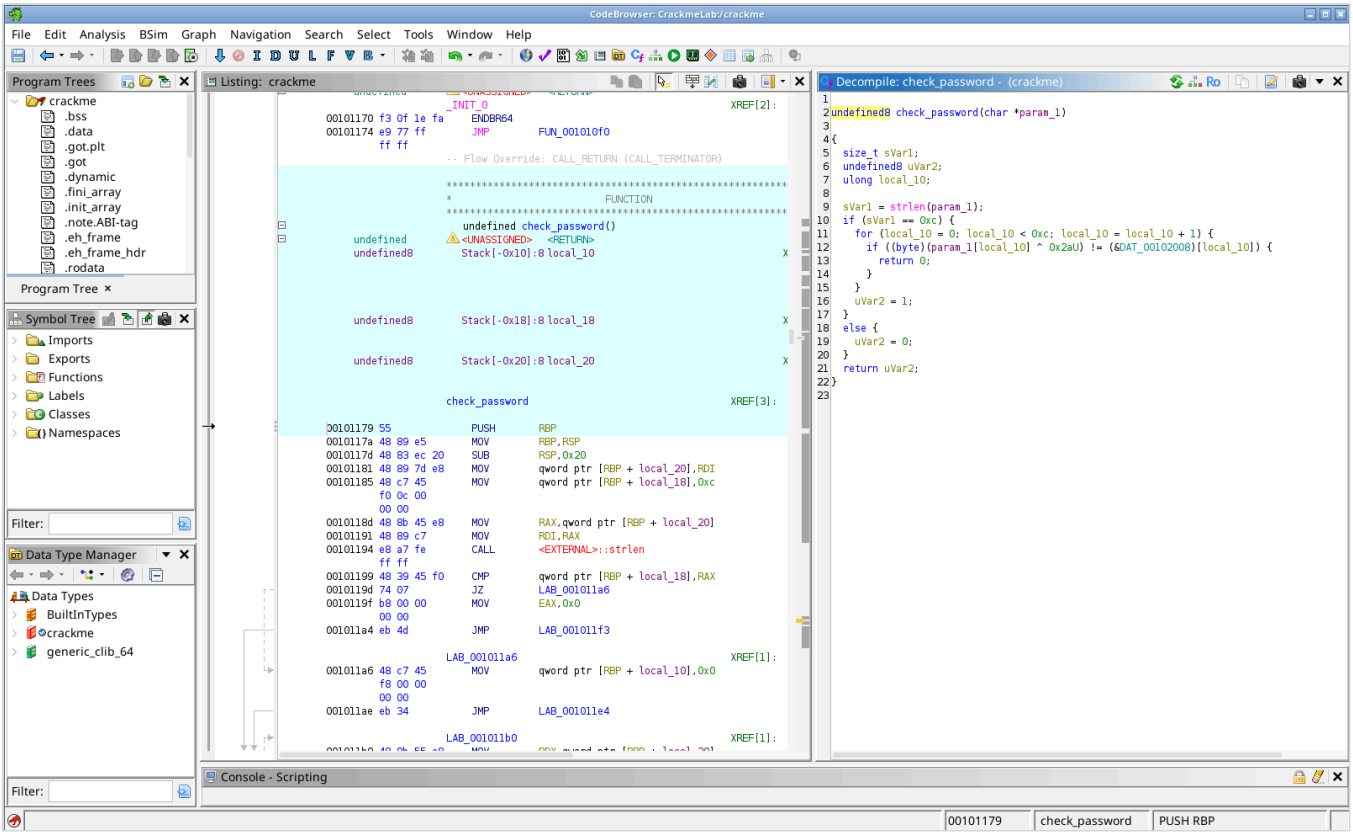

Real capture: rea decompile 0x101179, the password check

```
rea decompile $T 0x101179 --provider ghidra --json
```

The logic in plain English: the input must be exactly **12** characters (`0xc`), and for each position `input[i] XOR 0x2A` must equal byte `i` of a data array at **`0x102008`**. That's a classic XOR-obfuscated

secret, and `0x102008` is 9 bytes before the prompt string at `0x102011`. That's where the stray `IAY` came from.

Here's the same function in the Ghidra GUI, from my earlier manual walkthrough of this binary, for comparison:




Ghidra 12.1.4 GUI on the same crackme: the check function at 0x00101179

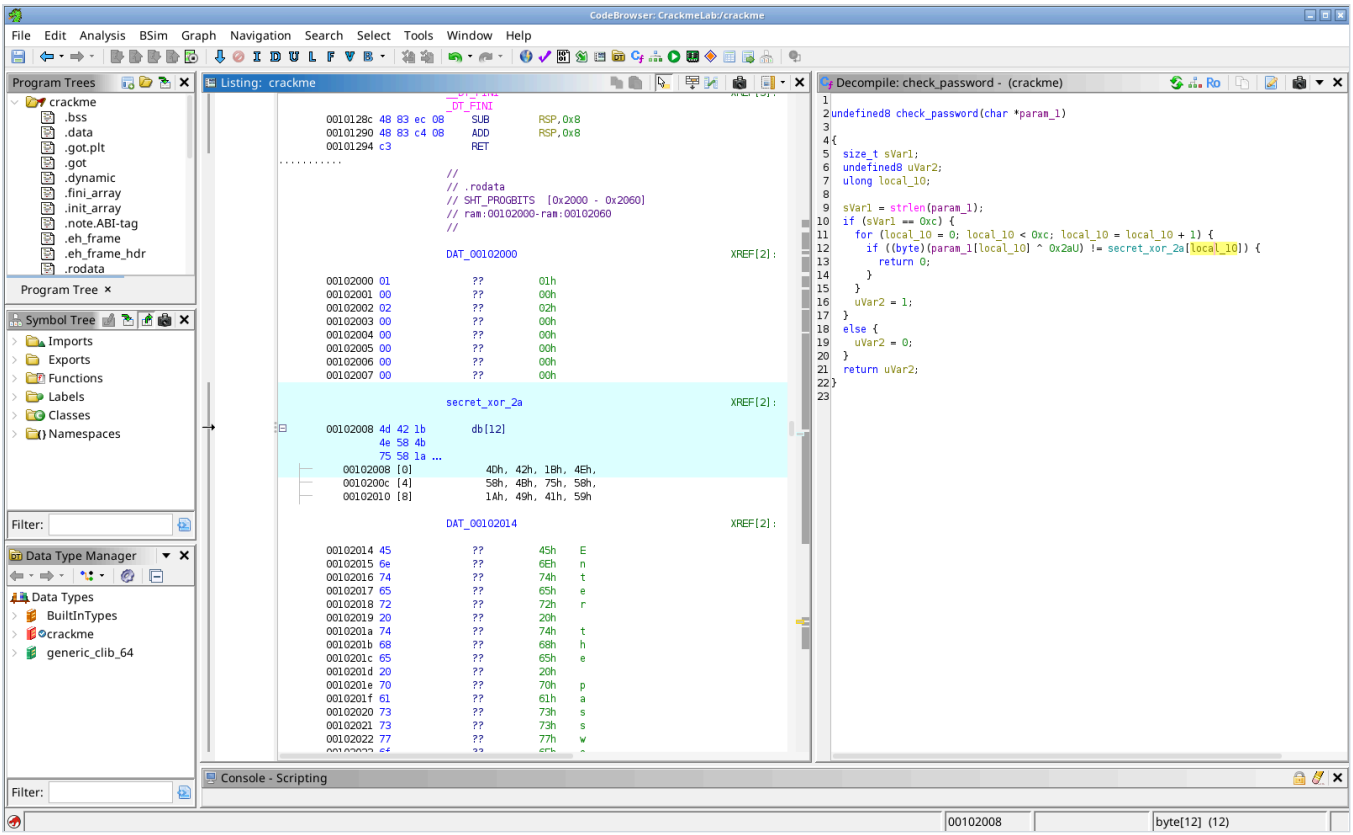
## Step 5: read the exact bytes




Real capture: rea read-bytes 0x102008, 12 bytes

```
rea read-bytes $T 0x102008 --length 12 --provider ghidra --json
```

Result: `4d421b4e584b75581a494159`, `returned_bytes: 12`, `complete: true`, `confidence: observed`. One beginner trap I hit: without `--length` you get 256 bytes requested and 89 returned, because the read *"stop[s] at a segment boundary."* Always ask for exactly what you need.




Ghidra GUI: the same 12-byte array at 0x00102008

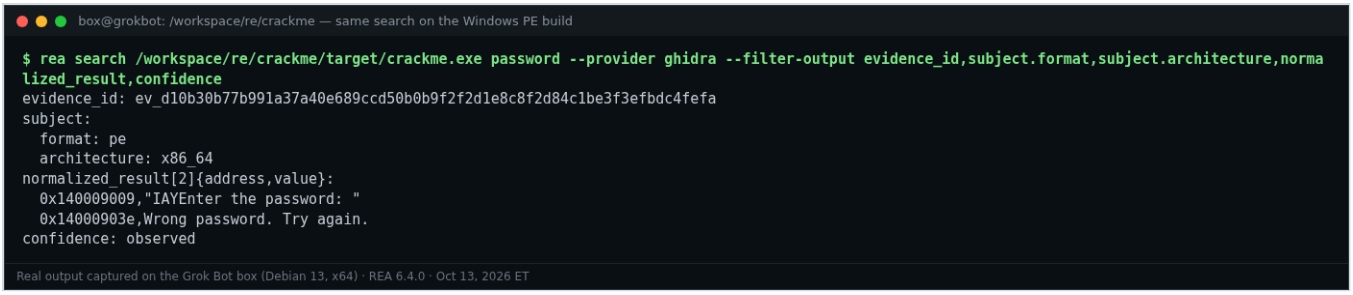
## Step 6: decode, then verify by running your own binary




Real capture: XOR-decode and run the crackme with the recovered password

XOR each byte with `0x2A` and you get a 12-character password. Spoiler: it's in the capture above. Running **my own** binary with it prints *"Access granted. Nice work!"* with exit code 0, and a wrong guess exits 1. This is the only step that executed the target, and it's labeled as runtime evidence, separate from the static findings. That separation is what the skill means by *"never imply that static analysis observed execution."*

### Step 7: same question, Windows build




Real capture: the same search against crackme.exe (PE, x86-64)

Linux Ghidra handles PE files too. The same `rea search` on `crackme.exe` reports `format: pe`, image-base addresses (`0x140009009`), and the same `IAY` tell. On a Windows host, this is exactly the read-only P0 boundary from §4.4.

**Total time** from a working install to the answer: six CLI commands, about a minute of wall-clock time. The habit matters more than the speed: string → reference → function → data → verification, with an Evidence ID at every hop.

---

## 8. The same investigation through the agent (MCP)




Real capture: the MCP tool results from my agent's REA session, projected with jq

The CLI is great for learning. Day to day, your agent drives REA over MCP and keeps **one session** open, so it doesn't re-import the binary for every question. Through the Grok Bot `rea` connector, I made these real calls on the same file:

1. `open_binary` with `{"path": ".../target/crackme", "provider_id": "ghidra"}` returned `format: elf`, `architecture: x86_64` and the same SHA-256.
2. `trace_feature` with `{"query": "Access granted"}` matched `0x10202b`, referenced from `0x101260` and `0x101267` in `FUN_001011f5` (`ev_83a9d141…`).
3. `xrefs` with `{"address": "0x102008"}` showed that only `0x1011c3` and `0x1011d1` reference the secret array, both inside the check function (`ev_2a61d6e1…`).
4. `annotate_native_function` renamed `0x101179` to `check_password` and attached a comment. Ghidra re-decompiled it with the new name, and REA's own limitation applies: *"original executable bytes are never written."*
5. `record_unknown` logged an open question (is there any other password path?) as `unk_c1d42d3c…`, severity low, with a recommended probe.
6. `export_evidence_bundle` wrote `notes/evidence.json`: **4 records, 1 unknown**.

One real snag worth knowing: my first `record_unknown` call failed with `evidence_integrity_mismatch`, because I cited an Evidence ID from a **CLI** run. CLI processes and the MCP session have separate ledgers. Cite IDs from the same session you're writing to.

---

## 9. The CONFIRMED/GUESS findings log and the read-only workspace




Real capture: my FINDINGS.md for the crackme, every claim tagged and receipted

The single best habit I picked up comes from a community guide by @zodchiii ("How to Turn Opus 5.5 Into Your Personal Reverse Engineer"), and it lines up with REA's own rules. **Every claim goes in a findings log tagged CONFIRMED or GUESS.**

**CONFIRMED** means the line cites an Evidence ID plus an address or `file:line`. **GUESS** means it doesn't yet, so it becomes a `record_unknown` entry with a probe that would settle it. My log has seven CONFIRMED lines (six static, one runtime on my own binary) and one GUESS that's still open as `unk_c1d42d3c…`. The xrefs result in §8 points toward "no second path," but I haven't closed it, because "points toward" isn't proof.

The workspace layout:

```
/workspace/re/<target>/
  target/   # chmod -R a-w : analysis input only, never edited
  notes/    # FINDINGS.md, ARCHITECTURE.md, evidence.json, spec/
```

Why make `target/` read-only at the **filesystem** level? Agent permission settings (like Claude Code's allow/deny lists) are useful, but they aren't a security boundary. A `Permission denied` from the OS is (§7, step 0). REA already never writes target bytes. The read-only folder protects you from *everything else*: a stray script, an over-eager agent, or you at 2 a.m.

Five failure modes to avoid, from the same guide: skipping the CONFIRMED/GUESS tags; analyzing raw HARs that still contain tokens (redact first); letting the model write to `target/`; opening a fresh session for every question; and trusting AI renames without spot-checking about 10% of them against the pseudocode.

---

# 10. Where REA gives you the biggest wins: a tour of the tool groups

box@grokbot: ~ — rea providers (capabilities + side effects)

```
$ rea providers 2>&1 | head -40
open: false
capabilities[18]:
  - operation: capture_native_ui_scenario
    available: false
    reason: Native macOS utilities require macOS.
    availability_code: unsupported_host
    effects:
      mutates_artifact: false
      launches_process: true
      may_show_ui: true
      may_access_network: true
      may_write_filesystem: true
      changes_permissions: false
      requires_root: false
    limitations[1]: Availability and textual formats depend on the installed macOS/Xcode toolchain.
  - operation: decode_interface_builder
    available: true
    reason: null
    availability_code: null
    effects:
      mutates_artifact: false
      launches_process: false
      may_show_ui: false
      may_access_network: false
      may_write_filesystem: false
      changes_permissions: false
      requires_root: false
… (13 more lines in the saved transcript)
```

Real output captured on the Grok Bot box (Debian 13, x64) · REA 6.4.0 · Oct 13, 2026 ET

Real capture: rea providers lists every capability with its side effects

A beginner-friendly feature: `rea providers` (and `rea capabilities`) lists every operation along with whether it `launches_process`, `may_access_network`, `may_write_filesystem` or `mutates_artifact`, and why it's unavailable on your host. Read that before you run anything that isn't purely static.

## 10.1 Web pages and JS bundles (Chrome CDP on loopback)

box@grokbot: /workspace/re/lab — loopback CDP: list-browser-targets + analyze-web-bundle

```
$ curl -s http://127.0.0.1:9333/json/version | jq -r .Browser
Chrome/154.0.8037.97

$ rea list-browser-targets http://127.0.0.1:9333 --allowed-origins http://127.0.0.1:8765 --filter-output evidence_id,normalized_result.targets,normalized_result.excluded
evidence_id: ev_553ef0397aa1424b84dc8e7dbf3cdb1b384be7d35e742fcad609a211e2c57465
normalized_result:
  targets[1]{target_id,type,title,url,origin,attached}:
    41DAC25A4D76D9793754752B1EEE9BC4,page,REA web demo (my own page),"http://127.0.0.1:8765/","http://127.0.0.1:8765",false
  excluded:
    disallowed_origin: 0
    unsupported_url: 0
    non_page: 2

$ rea analyze-web-bundle http://127.0.0.1:9333 41DAC25A4D76D9793754752B1EEE9BC4 --allowed-origins http://127.0.0.1:8765 --observation-ms 1500 --json | jq -c '.evidence_id, (.normalized_result.observations.chunks.nodes[] | {url, bytes}), (.normalized_result.observations.chunks.edges[] | {kind, specifier, line: .location.line}), {endpoints: .normalized_result.observations.endpoints, routes: .normalized_result.observations.routes}, .normalized_result.completeness'
"ev_65973234564924ffa793fcfbfc54e3a8db4b29cd2ac8f5ad9e681c806f9a69dc"
{"url":"http://127.0.0.1:8765/app.js","bytes":271}
{"url":"http://127.0.0.1:8765/export.js","bytes":132}
{"kind":"static_import","specifier":"./export.js","line":1}
{"endpoints":[],"routes":[]}
{"status":"complete","parsed_scripts":2,"parse_failures":0,"visited_ast_nodes":76}
```

Real output captured on the Grok Bot box (Debian 13, x64) · REA 6.4.0 · Oct 13, 2026 ET

Real capture: list-browser-targets + analyze-web-bundle on my own local page

I served a two-file demo page (`app.js` imports `export.js`) on `127.0.0.1:8765` and started headless Chrome with a **loopback** debugging port. Then:

```
rea list-browser-targets http://127.0.0.1:9333 --allowed-origins http://127.0.0.1:8765
rea analyze-web-bundle http://127.0.0.1:9333 <targetId> --allowed-origins http://127.0.0.1:8
765 --observation-ms 1500 --json
```

REA listed only the page matching my exact origin (it excluded the browser's other targets), captured both scripts with SHA-256s, and mapped the `static_import` edge from `app.js` line 1. The receipt: it reported **`endpoints: []`**, even though my code POSTs to `/api/v1/exports`. The URL is built inside an object and passed around, and REA's limitations say so honestly: *"String-built routes and endpoints... may remain unknown."* An empty result here doesn't mean "no endpoint." It means "not provable statically." **Close the gap with a saved capture (runtime evidence).** I recorded a HAR of *my own* page while clicking "Export CSV" (no cookies or auth headers in it), then ran REA's offline capture inspector:

```
rea inspect-web-network-capture /workspace/re/lab/web/demo.har har --json
```




Real capture: inspect-web-network-capture finds the POST that static analysis couldn't prove

Record 3 is the missing piece: **`POST http://127.0.0.1:8765/api/v1/exports`** with the body `{"format":"csv","range":"30d"}` (my toy server answered 501, which is expected). REA marks it `confidence: observed` but `runtime_attribution: "unknown"`, and its limitations say *"Recorded URLs are never fetched"* and *"Known authentication headers/cookies... are excluded."* Static analysis gives you the structure; a capture of your own session gives you the actual request. Use both, and label which is which. Before you analyze a real HAR, strip tokens yourself. REA's `--sensitive-value` flag lets you mark extra literal values as sensitive.

Version 6.4.0 also added tRPC, HTTP-link and named GraphQL operation discovery (release notes). Only point this at pages you're entitled to use, in your own logged-in session.

## 10.2 Electron and JavaScript apps (no disassembler)




Real capture: analyze-javascript-application on my demo Electron app

This is where REA saves beginners the most time. If the app is Electron, **don't open a disassembler**. I wrote a five-file Electron-style app (main, preload, renderer, an IPC handler) and ran:

```
rea analyze-javascript-application /workspace/re/lab/jsapp --json > jsapp-evidence.json
```

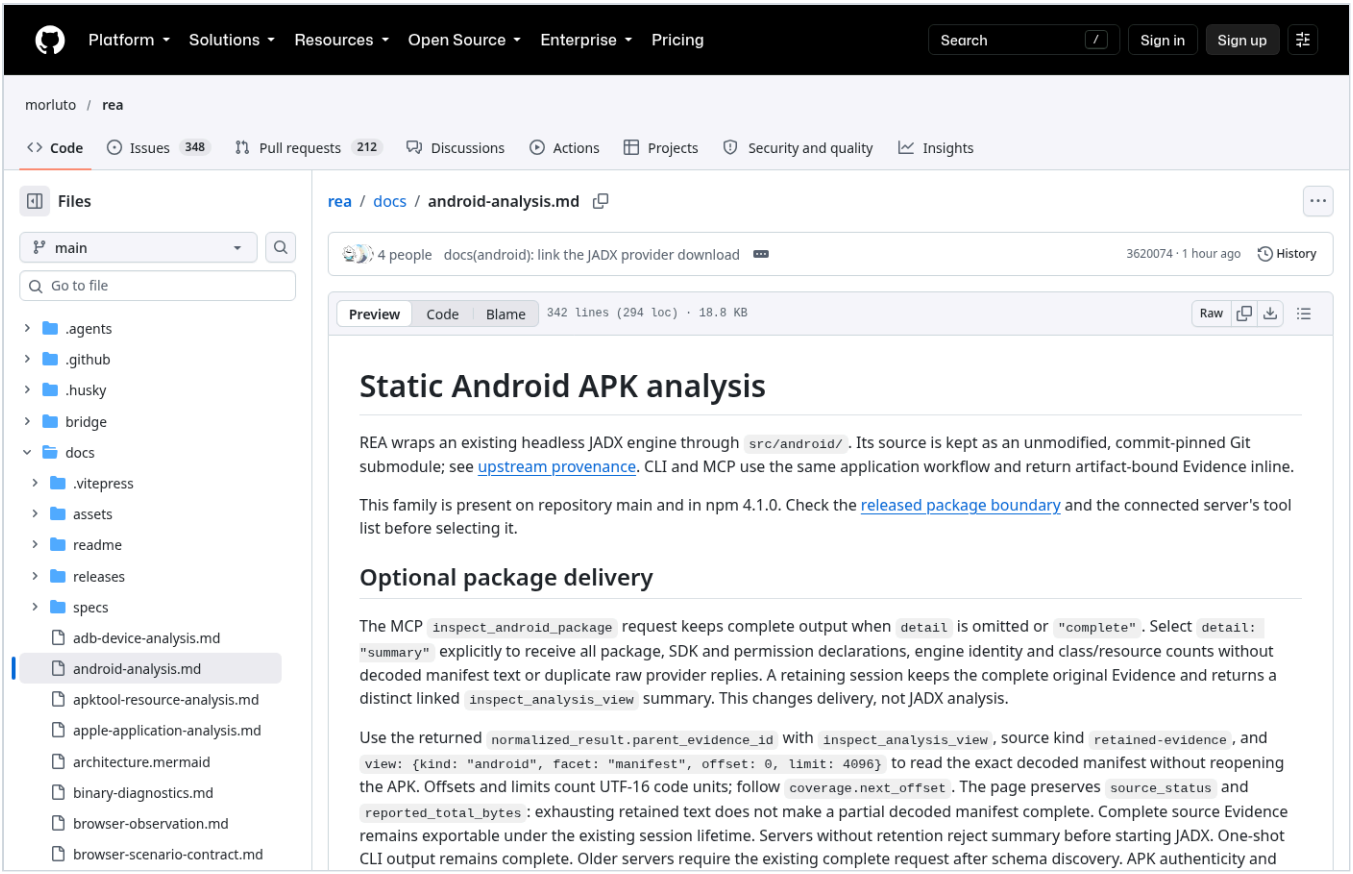
In under 2 seconds, it found 1 BrowserWindow, 1 preload entrypoint, 1 `contextBridge` API (`notes`, member `copy`), and the IPC channel **`note:copy`** with its main-process handler, paired with the renderer's call:




Real capture: the Electron boundary nodes REA recovered

Look at **`sender_validation_observations: 0`**. My `ipcMain.handle` never checks who sent the message. That's the kind of thing you'd flag in a security review of your own app. It's the same technique as REA's Notion clipboard showcase: renderer → preload → IPC → main. For real apps, the CLI guide says a 334 MB Obsidian Evidence file projects to a ~10 KB summary with `inspect-analysis-view`.

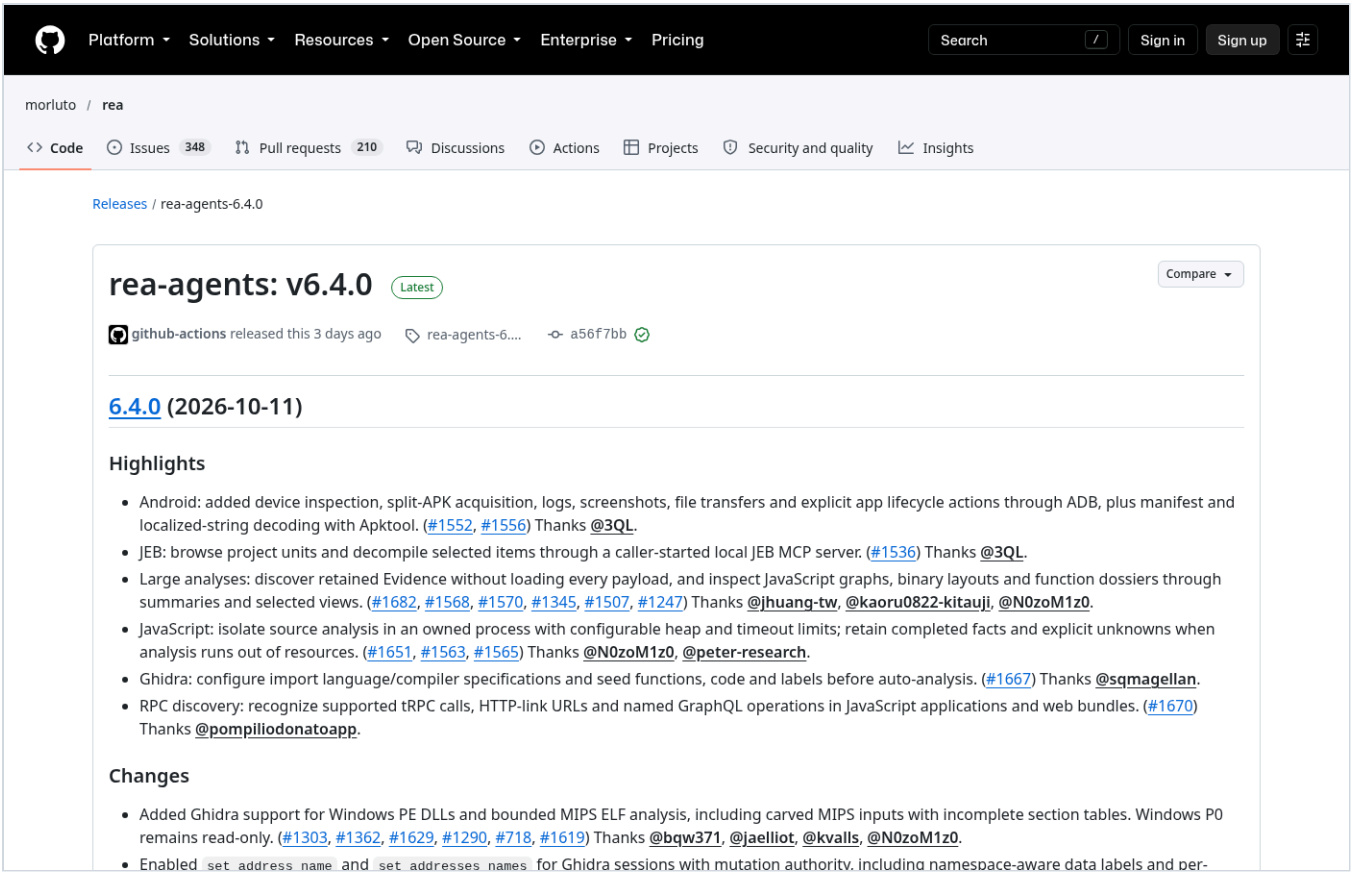
## 10.3 Android




REA docs: Static Android APK analysis

The Android tools are `inspect_android_package` (manifest, SDK, permissions, class counts), `search_android_classes`, `inspect_android_method` (decompile one method), `trace_android_references`, plus ADB device tools added in 6.4.0. They need a caller-supplied headless JADX (`REA_JADX_MCP_JAR`, jadx-headless-mcp 0.7.1) and a full JDK. **I didn't run these.** My box has no JADX, no adb and no APK I'm authorized to analyze, so I'm showing the docs instead of faking output. Guide: https://github.com/morluto/rea/blob/main/docs/android-analysis.md. Device tools that change state (`install_adb_package`, `push_adb_file`, `start_adb_app`) are flagged as side effects. Use them only on your own device or emulator.

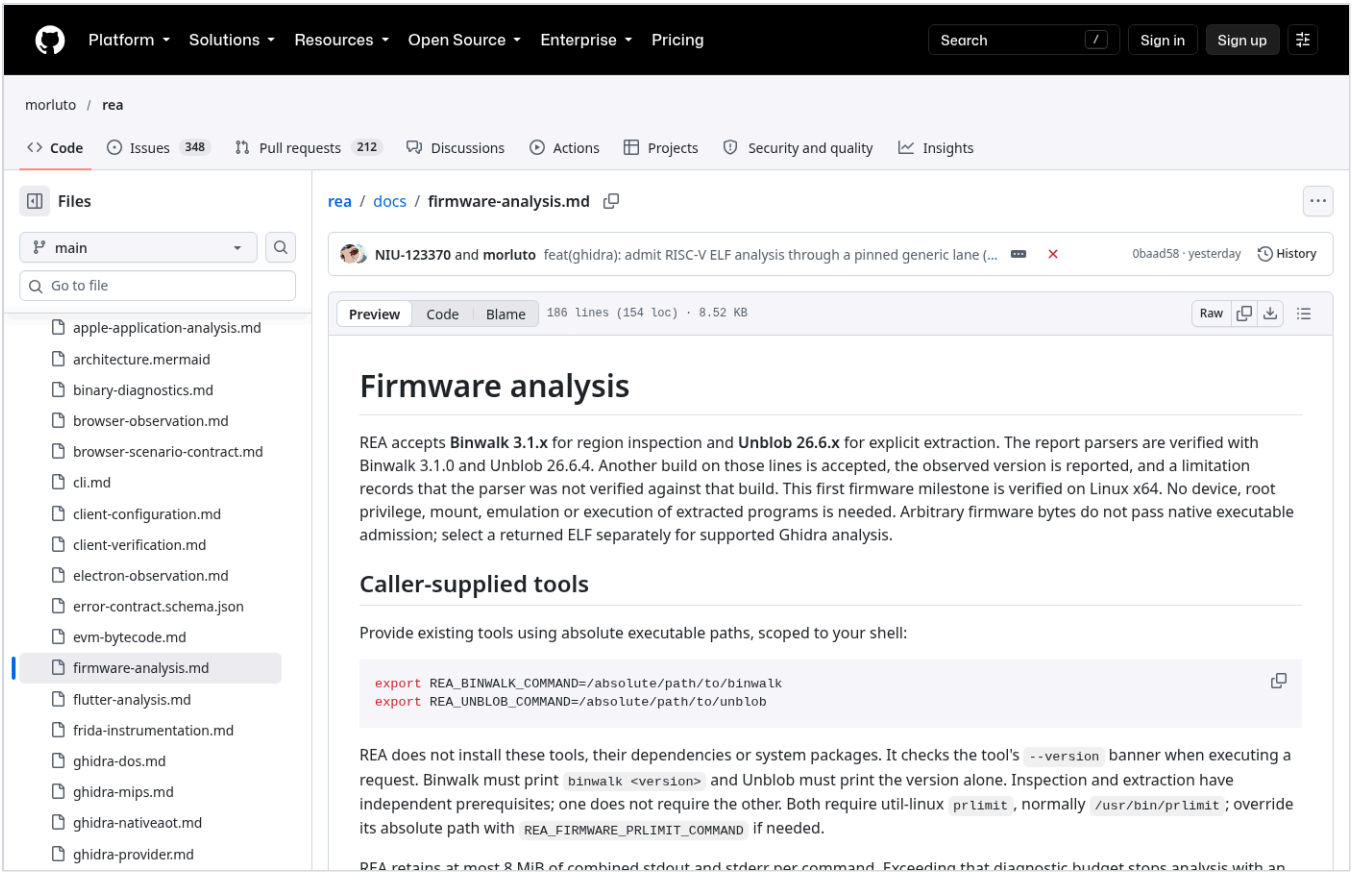
## 10.4 Native binaries (ELF, PE, Mach-O)




REA 6.4.0 release notes: Ghidra PE DLLs, MIPS ELF, import language and seeds

You've already seen the core flow in §7: `open_binary` → `search_strings` / `trace_feature` → `xrefs` → `procedure_pseudo_code` / `analyze_function` → `read_bytes` → `annotate_native_function` → `export_evidence_bundle` → `close_binary`. Beyond that there are call graphs (`get_call_graph`, `build_call_path`), `inspect_pe_resources` for Windows resources, Apple extras (`inspect_macho`, `get_objc_classes`, `analyze_swift_types`), and Ghidra import controls added in 6.4.0 (`REA_GHIDRA_LANGUAGE_ID`, seed files). Hopper is the default on macOS, where setup can install it with approval. IDA plugs in through an existing IDA MCP. Guide: https://rea.tools/guides/native/.

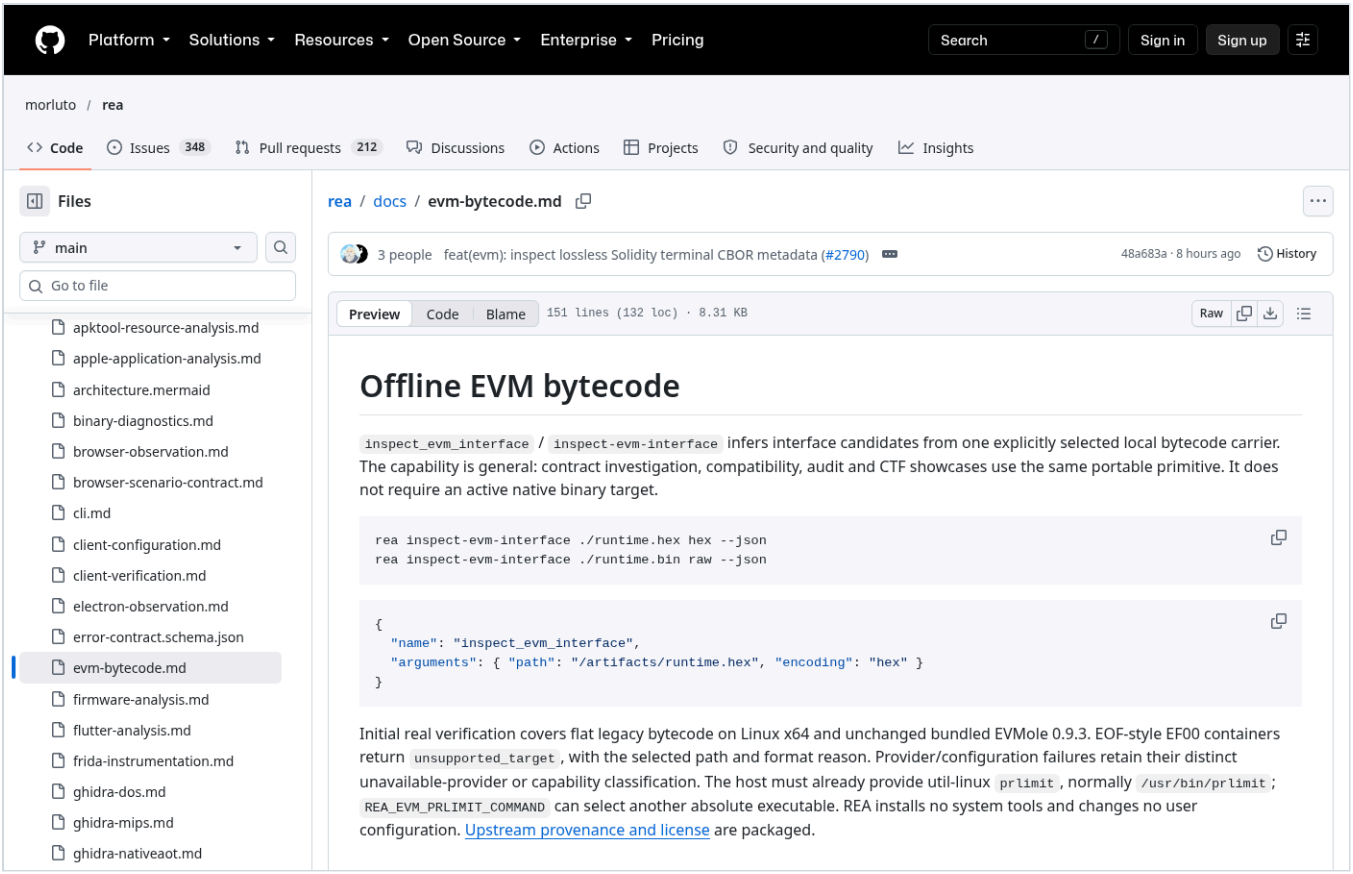
## 10.5 Firmware




REA docs: Firmware analysis with Binwalk/Unblob

`inspect_firmware_regions` lists signature candidates (offsets, lengths, descriptions) using caller-supplied **Binwalk 3.1.x**. `extract_firmware` extracts with **Unblob 26.6.x** into a new directory you choose. Then you `open_binary` the extracted images and continue with the native flow. The docs say *"This first firmware milestone is verified on Linux x64."* **I didn't run it**, because Binwalk and Unblob aren't installed on my box. Guide: https://github.com/morluto/rea/blob/main/docs/firmware-analysis.md.

## 10.6 Smart contracts (EVM bytecode, offline)




REA docs: Offline EVM bytecode

I hand-assembled a 62-byte runtime that dispatches two well-known selectors, `0xa9059cbb` (ERC-20 `transfer(address,uint256)`) and `0x70a08231` (`balanceOf(address)`), and ran:

```
rea inspect-evm-interface /workspace/re/lab/evm/demo-runtime.hex hex --json
```




Real capture: inspect-evm-interface recovers both selectors, as inferences

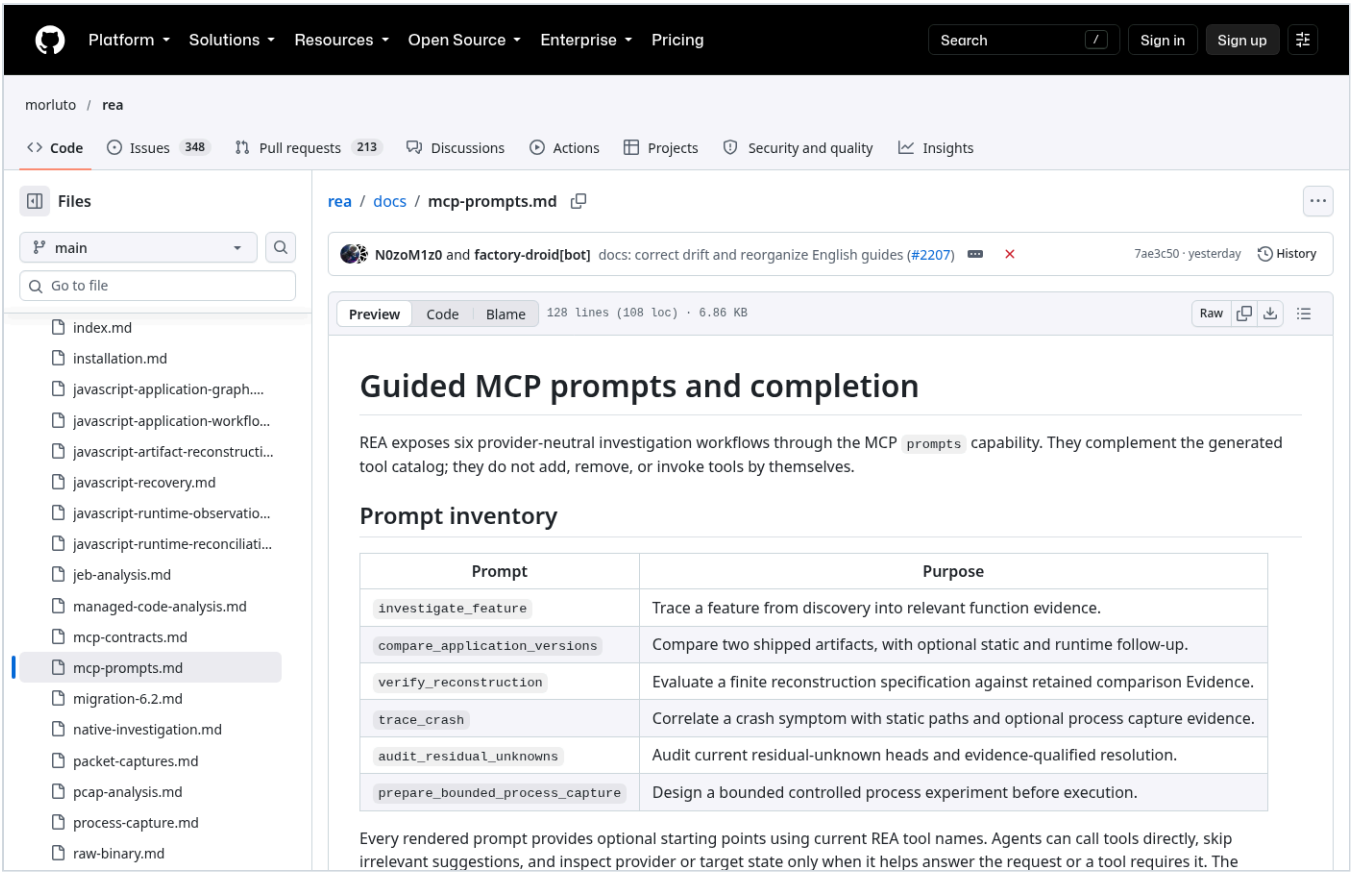
It found both selectors at the right byte offsets (29 and 49), with `runtime_execution: not-performed` and `discovery_completeness: unknown`. The lesson is in the details. It inferred `transfer` arguments as `uint256,uint256`, not `address,uint256`, because my toy code never masks the address. It labeled `balanceOf` **payable**, which is *correct for my bytecode*, since I left out the `CALLVALUE` check. That's exactly why the docs say selectors, arguments and mutability *"are inferences."* No chain, RPC or wallet is touched. It's audit support, not a trading bot.

### 10.7 .NET and the rest

`inspect_managed_artifact` and `inspect_managed_members` read .NET metadata and CIL without loading the assembly (guide). I had no .NET assembly to test on, so this one's docs-only. The 6.4.0 CLI also has commands for saved HAR/mitmproxy captures (`inspect-web-network-capture`), recorded Linux crashes (`inspect-recorded-crash`) and PE resources (`inspect-pe-resources`). The README on `main` lists more, such as SQLite snapshots and Flutter APKs. Those aren't all in 6.4.0 (there's no `inspect-sqlite-database` in my 6.4.0 `rea --help`). Check the generated tool catalog against your own server.

---

## 11. Five ready-to-paste prompts




REA docs: the six built-in MCP prompts

REA ships six guided MCP prompts: `investigate_feature`, `compare_application_versions`, `verify_reconstruction`, `trace_crash`, `audit_residual_unknowns` and `prepare_bounded_process_capture` (docs). These five paste-ready versions build on them. Replace the brackets.

#### 1. Route and map (start every job here)

```
Using REA, I want to understand how [FEATURE] works in [APP] at [ABSOLUTE PATH].
I own it / am authorized to analyze it. First tell me what kind of target this is and
which REA tool you'll open it with (route the target first). Static only, no execution.
Give me a one-paragraph map, keeping observations, inferences and unknowns separate,
with Evidence IDs.
```

### 2. Investigate one feature (native)

```
Run investigate_feature for "[FEATURE TEXT]" in [PATH] with provider ghidra.
Path: search_strings → trace_feature → xrefs → procedure_pseudo_code → read_bytes.
Log every claim in notes/FINDINGS.md as CONFIRMED (Evidence ID + address) or GUESS
(+ record_unknown). Stop after 30 functions and check direction with me.
Finish with export_evidence_bundle to notes/evidence.json and close_binary.
```

### 3. Electron / JS feature trace

```
Run analyze_javascript_application on [APP DIR OR .asar] with detail "summary", then
inspect_analysis_view. Trace "[FEATURE]" from the renderer through preload/contextBridge
and IPC to the main-process handler. Report channel names, handler locations and any
missing sender validation, with Evidence IDs. Don't execute the app.
```

### 4. What changed between versions

```
Run compare_application_versions on [OLD PATH] and [NEW PATH]. List behavior changes with
Evidence IDs, flag anything security-relevant, and record_unknown anything static
analysis can't settle. No execution.
```

### 5. Audit before you call it done

```
Run audit_residual_unknowns. List open unk_ items by severity, say which ones can be
resolved statically and with which tool, then mark each original question answered,
partially answered, or unresolved. Don't call the investigation complete while
required questions are open.
```

---

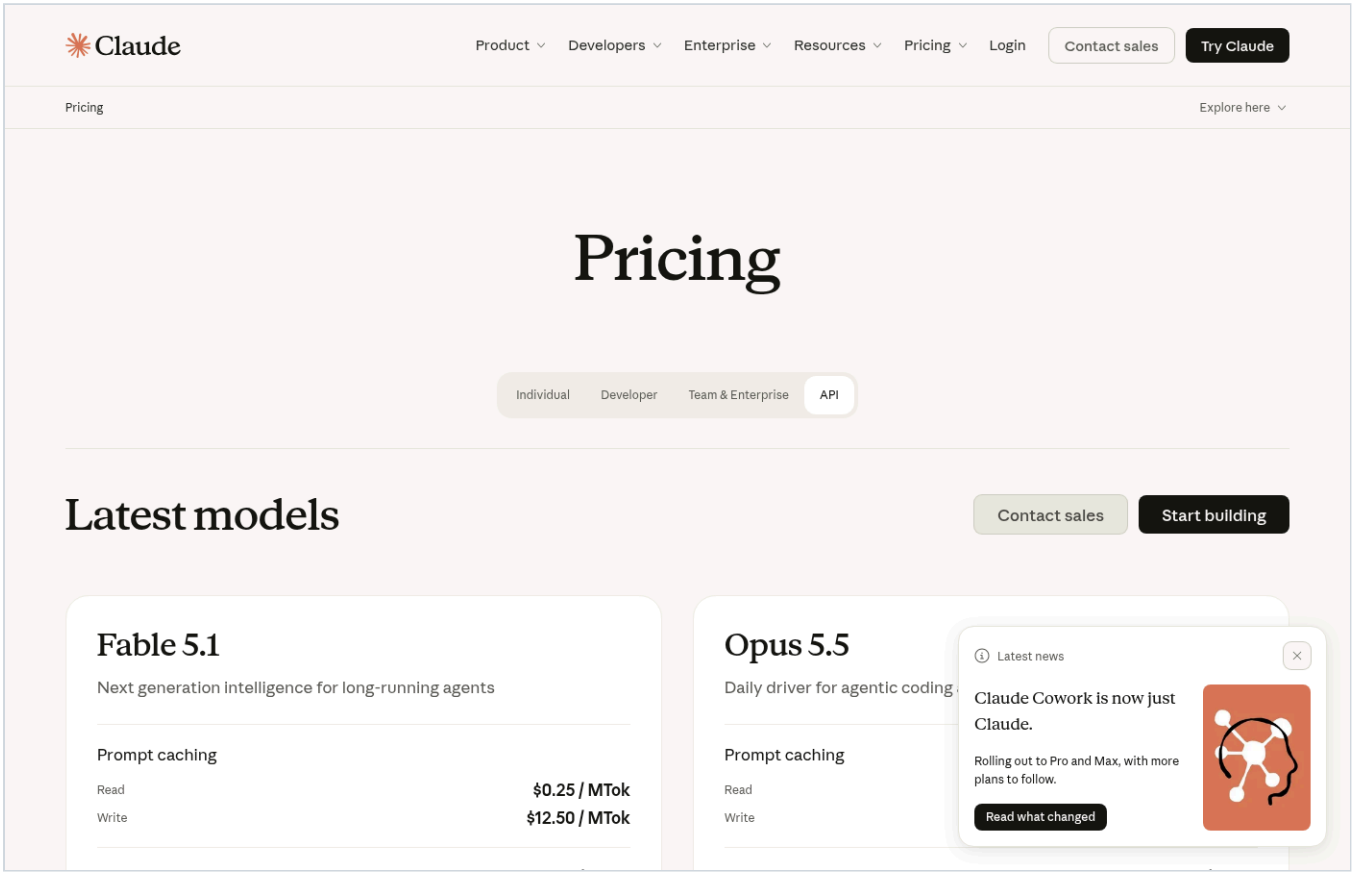
## 12. Hype vs receipts (short version)




Real capture: the GitHub API star count at 6:28 PM ET

| Claim going around | Receipt |
|---|---|
| "REA has 130K stars" (@zodchiii), "154K" (@woody_research) | **Outdated.** GitHub API: 204,103 at 5:43 PM ET (earlier project check) and **204,859 at 6:28 PM ET** (my capture), Oct 13. The README's own banner still says "180,000" |
| "Unlocked TradingView premium / free premium indicators" (@Bober_smart) | **Excluded.** That's paywall/license circumvention, which REA's DISCLAIMER explicitly rules out. I won't show it |
| "Opus 5.5: $5.60 vs $43.20 for a 400K-token RE session" | **Confirmed**, *within the 5-minute cache window* (see below) |

| Claim going around | Receipt |
|---|---|
| "$10M AI trading desk" video | **A paper-trading demo.** The dashboard is labeled *DESK · PAPER BOOK*. Nothing shows live returns |
| "List every tracker inside an APK" (@woody_research) | **Not a documented REA feature.** `inspect_android_package` reports declarations, permissions and classes. Spotting tracker SDKs is your own inference |
| `/mcp` shows tools named `decompile`, `search`, `trace` (@0xMovez course, quoted by @woody_research) | Those are **CLI** command names. The MCP tools are `procedure_pseudo_code`, `search_strings` and `trace_feature` |

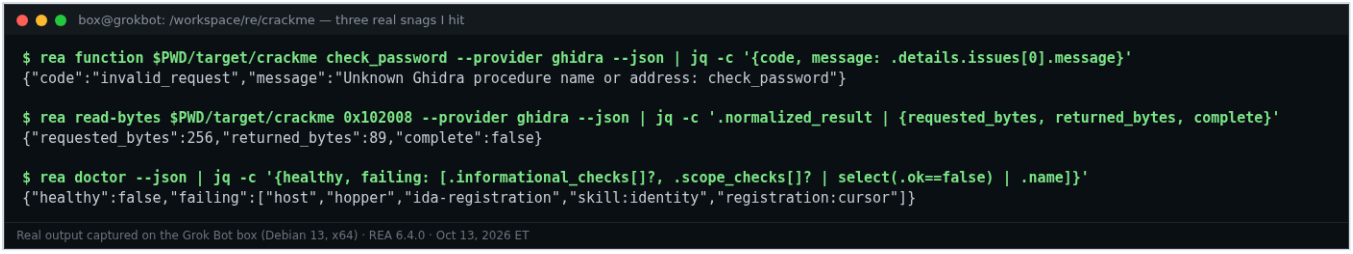



claude.com/pricing: Opus 5.5 API rates, prompt caching at a 5-minute TTL

**The cost math, recomputed.** Opus 5.5 lists $4/MTok input, $20/MTok output, $5/MTok cache write and $0.20/MTok cache read, and the page notes *"Prompt caching pricing reflects 5-minute TTL"* (claude.com/pricing). Take a 400K-token context, 25 follow-ups and 80K output tokens in total. Cached: $2.00 to write + 25 × $0.08 in reads = $2.00, plus $1.60 output, for **$5.60**. Fresh every time: 26 × $1.60 = $41.60, plus $1.60, for **$43.20**. Both numbers check out, but **only if each follow-up lands within 5 minutes of the last one**. Otherwise you pay the $2 write again. The practical lesson is the same as in §8: keep one warm session per target.

---

# 13. Troubleshooting (real snags and documented fixes)

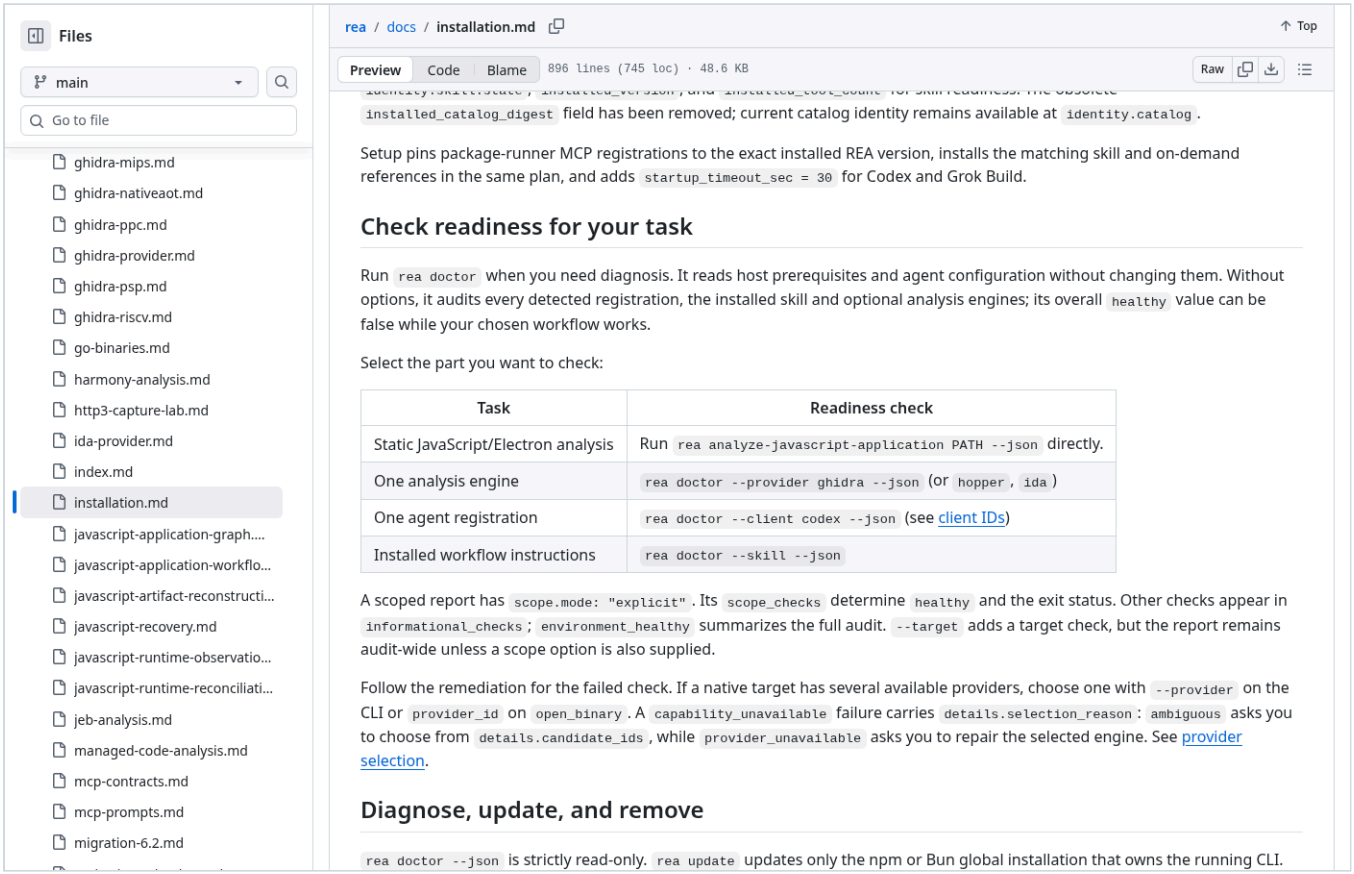



Real capture: three snags I actually hit, with REA's own error output

Every problem below either happened to me on this box (marked **hit**) or is a documented fix from REA's installation and client configuration guides (marked **docs**).

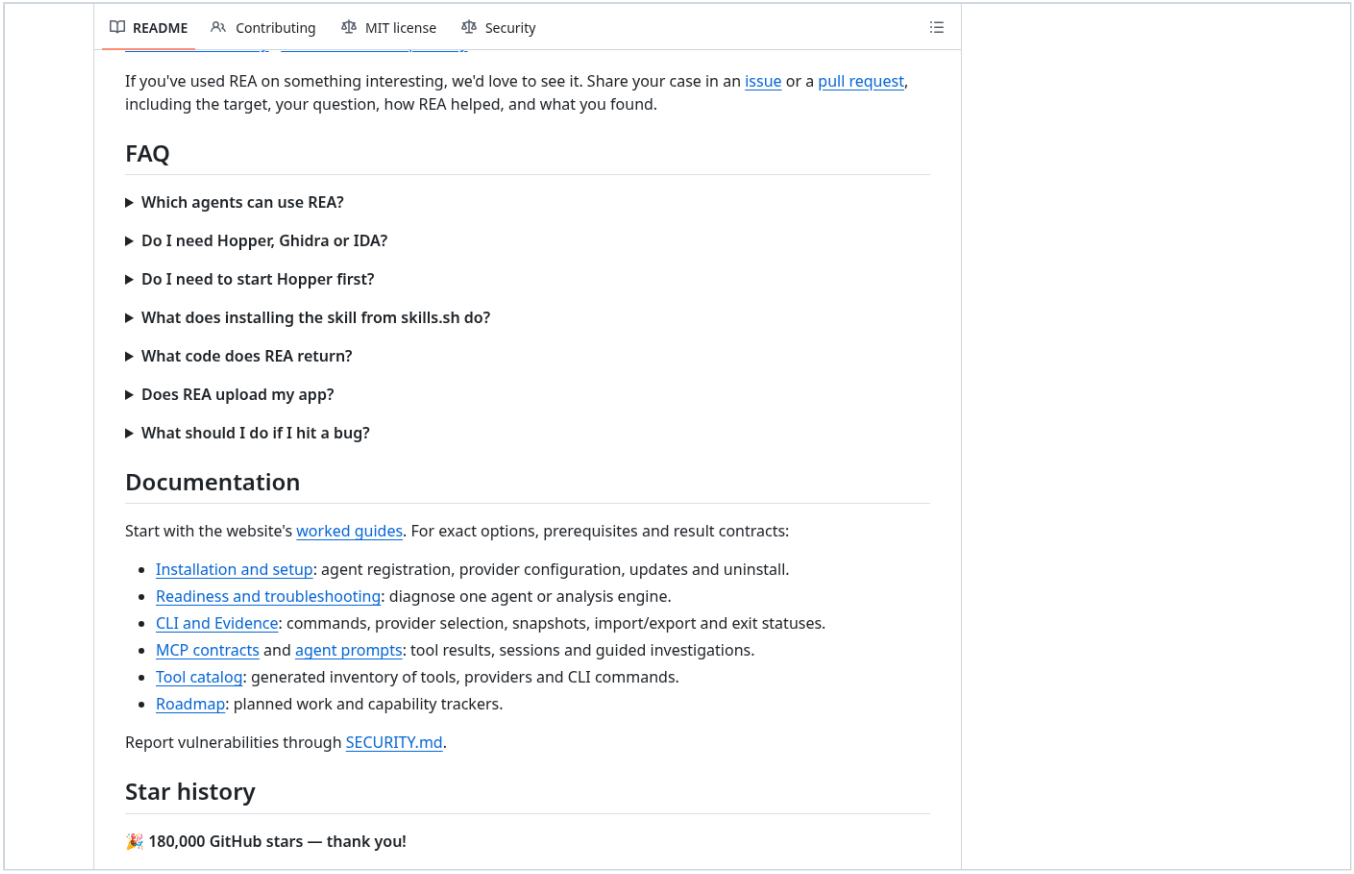
| Symptom | Cause | Fix |
|---|---|---|
| `Unknown Ghidra procedure name or address: check_password` (**hit**) | The binary is stripped. Your guessed name doesn't exist yet | Find the function with `rea search` / `rea trace` first, then use its **address** (`0x101179`). Rename it in an MCP session with `annotate_native_function` |
| `read-bytes` returns 89 of 256 bytes, `complete: false` (**hit**) | Reads stop at a segment boundary. The default length is 256 | Pass `--length N` with exactly what you need |
| `rea doctor` says `healthy: false` even though analysis works (**hit**) | Unscoped doctor audits *everything* (Hopper, IDA, every client, host list) | Scope it: `rea doctor --provider ghidra --json` or `--client cursor --json`. Read `scope_checks` |
| `record_unknown` fails with `evidence_integrity_mismatch` (**hit**) | You cited an Evidence ID from a different process (a CLI run) | Cite IDs produced in the same MCP session |
| `'inspect-go-binary' is not a command` (**hit**) | A main-branch feature that isn't in npm 6.4.0 | Use your server's tool list (`rea --help`, MCP `tools/list`). Wait for a release that includes it |
| Node 20 on PATH (**hit**) | Unsupported runtime (needs 22.19+/24.11+/26+, not 23/25) | Install a supported Node and put it first on PATH. Check with `node --version` |
| REA tools don't appear in the agent after setup (**docs**) | The agent wasn't restarted, or the registration is stale | Restart the agent. Run `rea doctor --client <id> --json`, then `rea setup --client <id> --dry-run --json` and approve the plan |
| Grok Bot shows no REA tools (**docs**) | Grok Bot doesn't read local `mcp.json` | In Grok Bot chat, add a custom MCP server named `rea` running `npx -y rea-agents@<version> mcp` |
| First Ghidra query times out on a big binary (**docs**) | Import plus auto-analysis exceeds the deadline (default 330,000 ms) | Set `REA_GHIDRA_STARTUP_TIMEOUT_MS` in the server environment, and raise your MCP client's own tool timeout |
| Ghidra runs out of memory (**docs**) | Default JVM heap is `2G` | Set `GHIDRA_HEADLESS_MAXMEM` (e.g. `4G`) in the environment that starts REA |
| `native_decompiler` missing on macOS (**docs**) | The Ghidra archive lacks the macOS decompiler binary | Build it with `make ghidra_opt` as shown in the installation guide (x64 and arm64 recipes) |
| Claude Desktop loses the registration (**docs**) | The running app rewrote its config (seen with the | Quit Claude Desktop **fully** before approving setup, then start it again |

| Symptom | Cause | Fix |
| --- | --- | --- |
| | Windows MSIX build) | |
| Claude Code "autocompact is thrashing" with a non-Anthropic endpoint (**docs**) | Tool search is off, so all ~170 tool schemas load upfront | Add `"REA_MCP_INPUT_SCHEMA_PROFILE": "compact"` to `mcpServers.rea.env` in `~/.claude.json` and restart |
| Hopper says `Cannot launch Hopper! Connection is invalid.` (**docs**) | A pending demo/license/loader dialog | Complete the dialog in the same user session, then retry |
| Anything else (**docs**) | Often already fixed upstream | Run `rea update` (global install) or `npx rea-agents@latest setup`, restart, retry. Then open an issue with your REA version, target type, steps and error output |




REA docs: scoped readiness checks, the fastest way to diagnose one task

# 14. FAQ



README Contributing MIT license Security

If you've used REA on something interesting, we'd love to see it. Share your case in an issue or a pull request, including the target, your question, how REA helped, and what you found.

## FAQ

- ▶ Which agents can use REA?
- ▶ Do I need Hopper, Ghidra or IDA?
- ▶ Do I need to start Hopper first?
- ▶ What does installing the skill from skills.sh do?
- ▶ What code does REA return?
- ▶ Does REA upload my app?
- ▶ What should I do if I hit a bug?

## Documentation

Start with the website's worked guides. For exact options, prerequisites and result contracts:

- Installation and setup: agent registration, provider configuration, updates and uninstall.
- Readiness and troubleshooting: diagnose one agent or analysis engine.
- CLI and Evidence: commands, provider selection, snapshots, import/export and exit statuses.
- MCP contracts and agent prompts: tool results, sessions and guided investigations.
- Tool catalog: generated inventory of tools, providers and CLI commands.
- Roadmap: planned work and capability trackers.

Report vulnerabilities through SECURITY.md.

## Star history

🎉 180,000 GitHub stars — thank you!

The README's own FAQ

**Is REA free?** Yes. REA-authored code is MIT-licensed. Ghidra is free too. Hopper, IDA and JEB are commercial and need your own license.

**Does REA upload my app?** No. From the README: *"REA analyzes targets locally. Your agent receives the tool results, and its model provider has its own data policy."* Treat tool results as data you're sending to your model provider.

**Do I need Ghidra, Hopper or IDA?** Only for native binaries. Static JavaScript/Electron, .NET, EVM and saved-capture analysis need no engine (README FAQ).

**Which agents work?** *"Any agent that supports local MCP servers."* Setup configures the supported list automatically (§5). Grok Bot uses a hosted connector you add in chat.

**Is it legal to use REA on app X?** REA can't answer that for you. Its DISCLAIMER says you must establish *"the legal authority and scope required"* yourself, and that reverse engineering *"may be restricted in your jurisdiction or by the license of the software you analyze."* If you're not sure, practice on your own builds and CTFs.

**Will it run the program?** Static tools don't. Runtime tools (process capture, browser scenarios, ADB app start) do, and `rea providers` lists each one's `launches_process` and other side effects. In this guide, the only execution was me running my own crackme.

**Does REA modify the binary?** No. With Ghidra it imports an ephemeral, read-only copy, and annotations live only in the session database.

**What does REA return: source code?** No. You get pseudocode and assembly for native code, and modules and relationships for JS. The README FAQ: *"Your agent uses these findings to write and test an implementation."* Pseudocode isn't the original source.

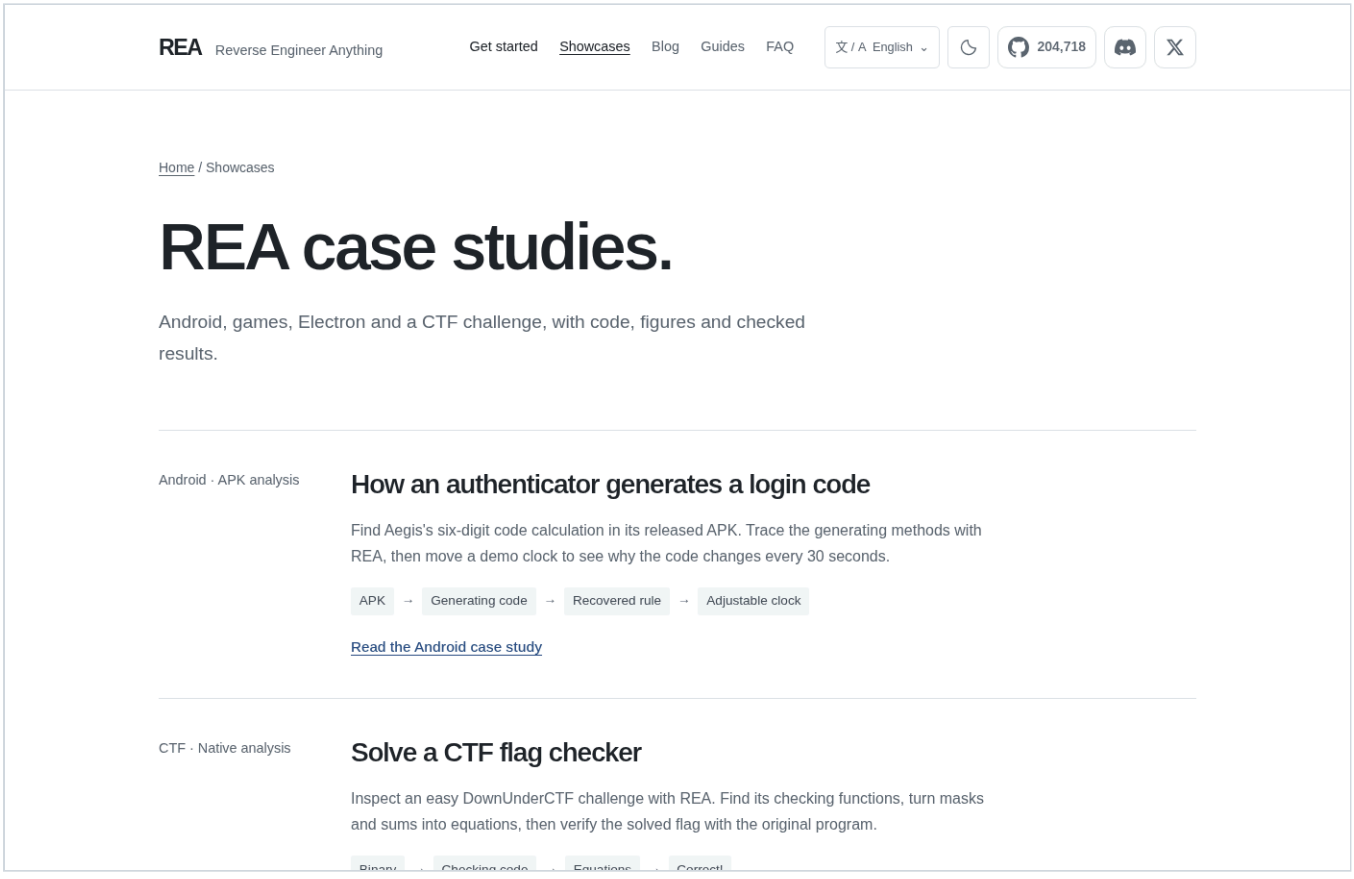
**Why do Evidence IDs repeat between runs?** They're content-addressed. Running the same search on the same file gave me the same `ev_bdafc8cd…` both times, which makes results easy to cite and check.

**What's the difference between `main` and npm?** `main` is newer than the release. Features documented on `main` (packet captures, Go metadata and more) may not be in npm 6.4.0. Check `npm view rea-agents dist-tags.latest` and your server's actual tool list.

**How do I update or uninstall?** `rea update` for global installs, or `npx rea-agents@latest setup` for package-runner setups. `rea uninstall` removes REA's registrations and skill files. `rea uninstall --purge-data` also removes `~/.rea` cache and state, while preserving Hopper, Node.js, Evidence files and other MCP servers.

---

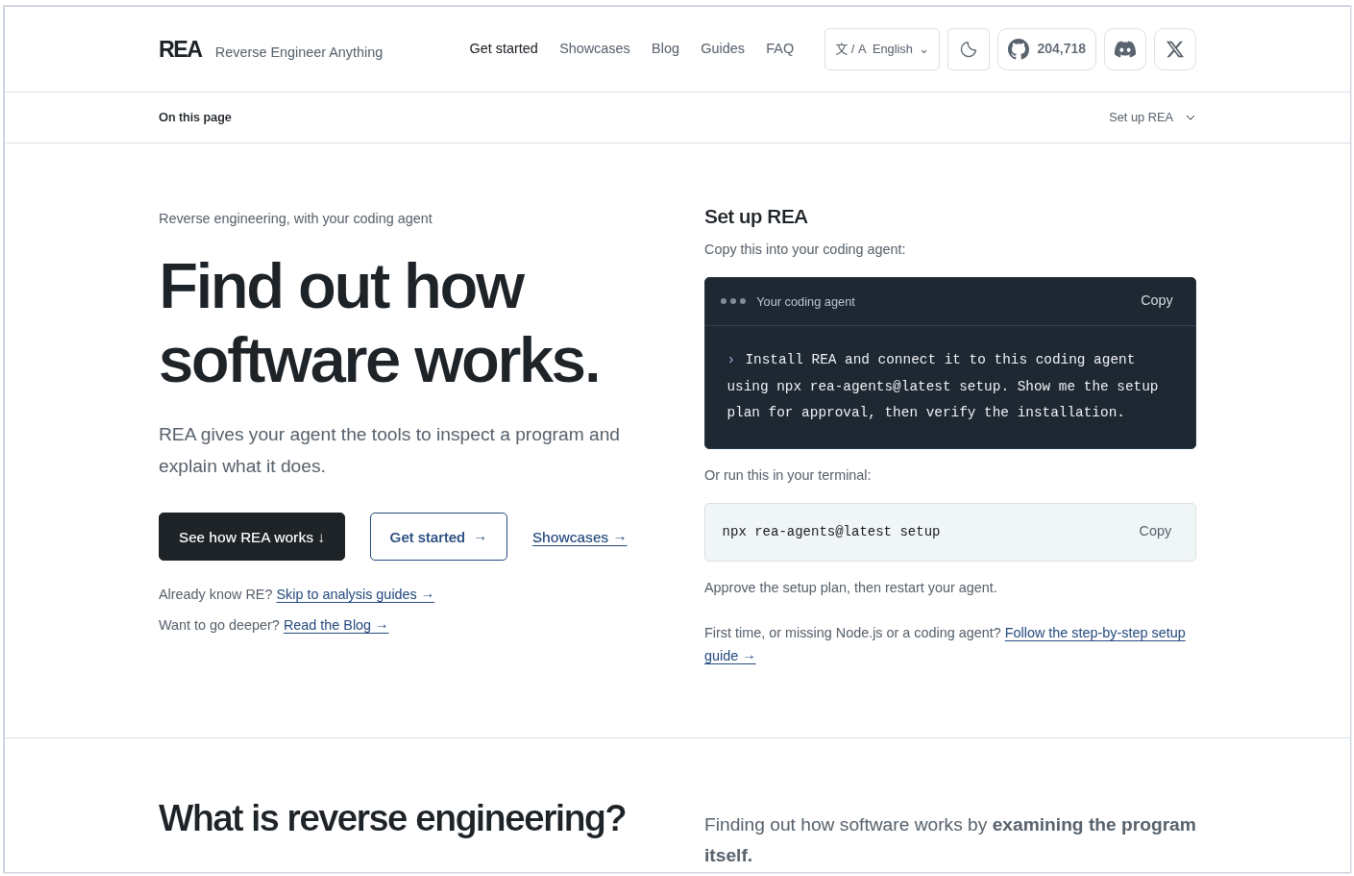
## 15. Next steps and links




REA case studies at rea.tools/showcase

1. Install Node in a supported range, run `npx rea-agents setup`, then restart your agent.
2. Add Ghidra 12.1.4 + JDK 21 and check with `rea doctor --provider ghidra --json`.
3. Build a crackme of your own (or grab a CTF binary you're allowed to use) and repeat §7 with a findings log.
4. Point `analyze_javascript_application` at an Electron app *you* wrote.

5. Read the skill, then the six MCP prompts, then one showcase end to end.
6. Run `rea update` (global installs) or `npx rea-agents@latest setup` regularly. The README says *"REA changes quickly."*




The REA website, rea.tools

### Official links (all checked Oct 13, 2026 ET)

| What | Link |
|---|---|
| Repo | https://github.com/morluto/rea |
| Release 6.4.0 (Oct 11, 2026) | https://github.com/morluto/rea/releases/tag/rea-agents-6.4.0 |
| Disclaimer | https://github.com/morluto/rea/blob/main/DISCLAIMER.md |
| Installation | https://github.com/morluto/rea/blob/main/docs/installation.md |
| Client configuration (Grok Bot, Claude, Cursor…) | https://github.com/morluto/rea/blob/main/docs/client-configuration.md |
| Agent skill | https://github.com/morluto/rea/blob/main/.agents/skills/reverse-engineer-anything/SKILL.md |
| CLI and Evidence | https://github.com/morluto/rea/blob/main/docs/cli.md |
| MCP prompts | https://github.com/morluto/rea/blob/main/docs/mcp-prompts.md |
| Windows Ghidra P0 | https://github.com/morluto/rea/blob/main/docs/windows-ghidra-p0.md |
| Website, guides, showcases | https://rea.tools/ · https://rea.tools/guides/ · https://rea.tools/showcase/ |
| Ghidra 12.1.4 | https://github.com/NationalSecurityAgency/ghidra/releases/tag/Ghidra_12.1.4_build |

*If you use REA on something interesting (and legal), the project asks you to share the target, your question, how REA helped and what you found in an issue or PR. I'd love to see it too. Reply with your receipts.*